SAPIENS
Decision

Decision Analytics (DAN) Installation and Configuration Guide

Document Version : 1.5
Software Version: 1.5
March 2026

Partnering for Success

Notice

Trademarks

Statement of Conditions

Caution

Any changes or modifications of the software not expressly approved in writing by the manufacturer could void the user's authority to use the application and its warranty.

Document Revision History

Revision	Date	Author	Description	Approved by
1.0	August 2025	Decision Documentation	Describes the Decision Analytics (DAN) 1.0 Release	Decision Product Management
1.1	October 2025	Decision Documentation	Describes the Decision Analytics (DAN) 1.1 Release	Decision Product Management
1.1a	October 2025	Decision Documentation	Removed the **schema DDL** reference from the document	Decision Product Management
1.5	March 2026	Decision Documentation	Describes the Decision Analytics (DAN) 1.5 Release	Decision Product Management

Document Authorization

This document is digitally signed and authorized by the Sapiens Decision documentation team. The steps to authenticate the digital certificate are documented in the Index file present in the documentation package. In case of any queries or feedback on the document, send an e-mail to Decision_documentation@sapiens.com.

Table Of Contents

1. About this Document

This guide provides installation and configuration instructions for Decision Analytics (DAN).

2. System Requirements

The following are the optimum requirements to run the DAN server.

Note: Java JDK is not part of the application package. You must manually install the JDK recommended version mentioned in the table.

Software Requirements	
Java Application Server	Certified with Tomcat 10.1.52
Database	DAN schema on PostgreSQL DB – 14.6
Operating System (OS) with Java support	Certified with Red Hat Enterprise Linux 8 and 9.2, AWS Linux 2
Docker	19.x, 20.x
Kubernetes	Client Version: v1.28.4 Customize Version: v5.0.4-0.20230601165947-6ce0bf390ce3
Helm	3.14.4
Java JDK	Oracle – 17.0.14 Amazon-Corretto - 17.0.14 Red Hat OpenJDK – 17.0.14

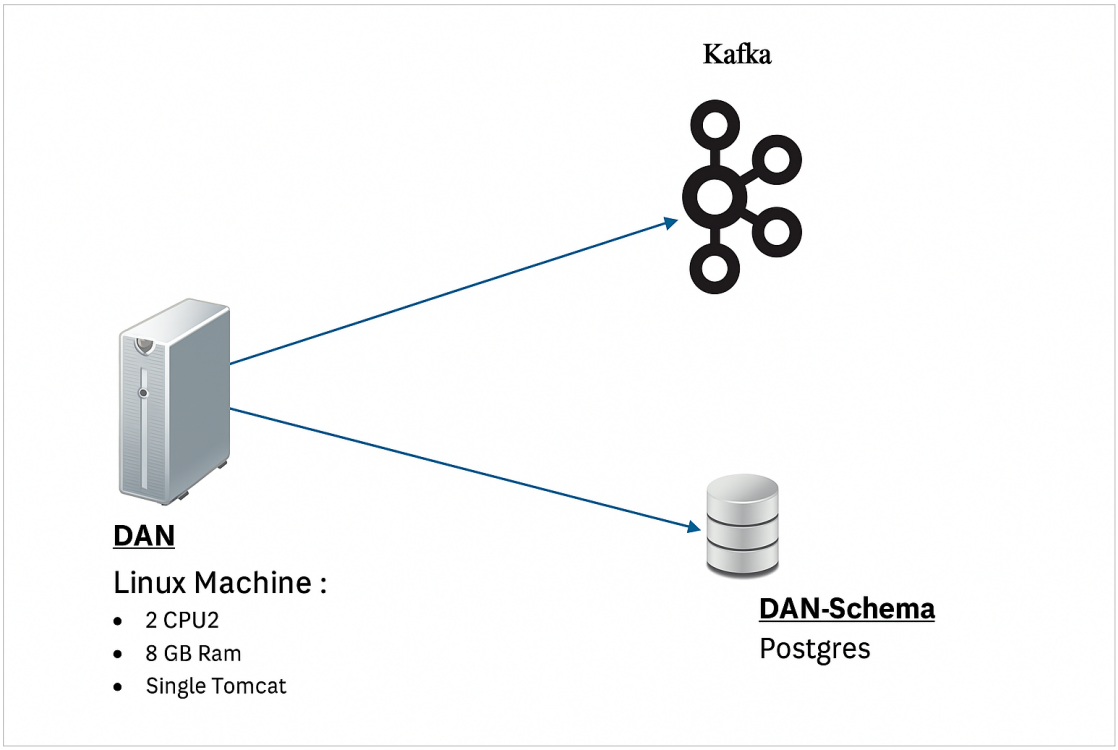
Hardware Requirements	
Hardware	**Recommended Storage**
CPU	2 CPU with 2.5 GHz or above
RAM	8 GB
Java heap size	4 GB - recommended
Hard disk	50 GB

3. Version Compatibility

For version compatibility information for the Decision Analytics (DAN) application, refer to the Version Compatibility section in the *Decision Analytics (DAN) Release Notes*.

4. Installation

This section provides step-by-step instructions for a base installation of the Decision Analytics (DAN). For advanced installation options such as Azure Key Vault integration, secured properties, custom context path, and HTTPS configuration, refer to the DAN Configuration section.



DAN– Single Instance Setup

Required Database Privileges

To install and operate DAN, the following RDBMS privileges must be granted to the DAN schema:

- CREATE / UPDATE / DELETE TABLE
- SESSION
- VIEW
- SEQUENCE

System Requirements and Access

To proceed with the installation, ensure the following prerequisites are met:

- Tomcat server access to the RDBMS
- Tomcat setup for HTTP or HTTPS
- Access to the deployed service

Installation Stages

The DAN installation process consists of the following stages:

1. **Download the Release Package:**

   Download the DAN release package from the Sapiens Azure storage location provided in the release notification email. If not received, please contact the Decision Support Team to request the download link.

   The release package contains:

   - Configuration files
   - WAR file for the DAN server

2. **Create the DAN Schema:**

   Create the required DAN schema in your relational database management system (RDBMS).

3. **Deploy WAR and Configuration Files**

   Deploy the following to your Tomcat server:

   - The WAR file
   - The configuration files (typically placed in the conf directory)

4. **Configure the Application**

   Update and configure the necessary conf files to complete the setup.

5. DAN Deployment & Installation

5.1 Overview

Decision supports multiple deployment styles, each distributed as a specific package. While the packages are functionally equivalent, they differ in:

- Layout
- Installation and configuration methods
- Platform dependencies
- Start up mechanisms and commands

5.2 Containerized Image Deployment

Containerized deployment packages the application components as Docker images, enabling flexibility, scalability, and simplified management across containerized platforms.

5.2.1 Overview

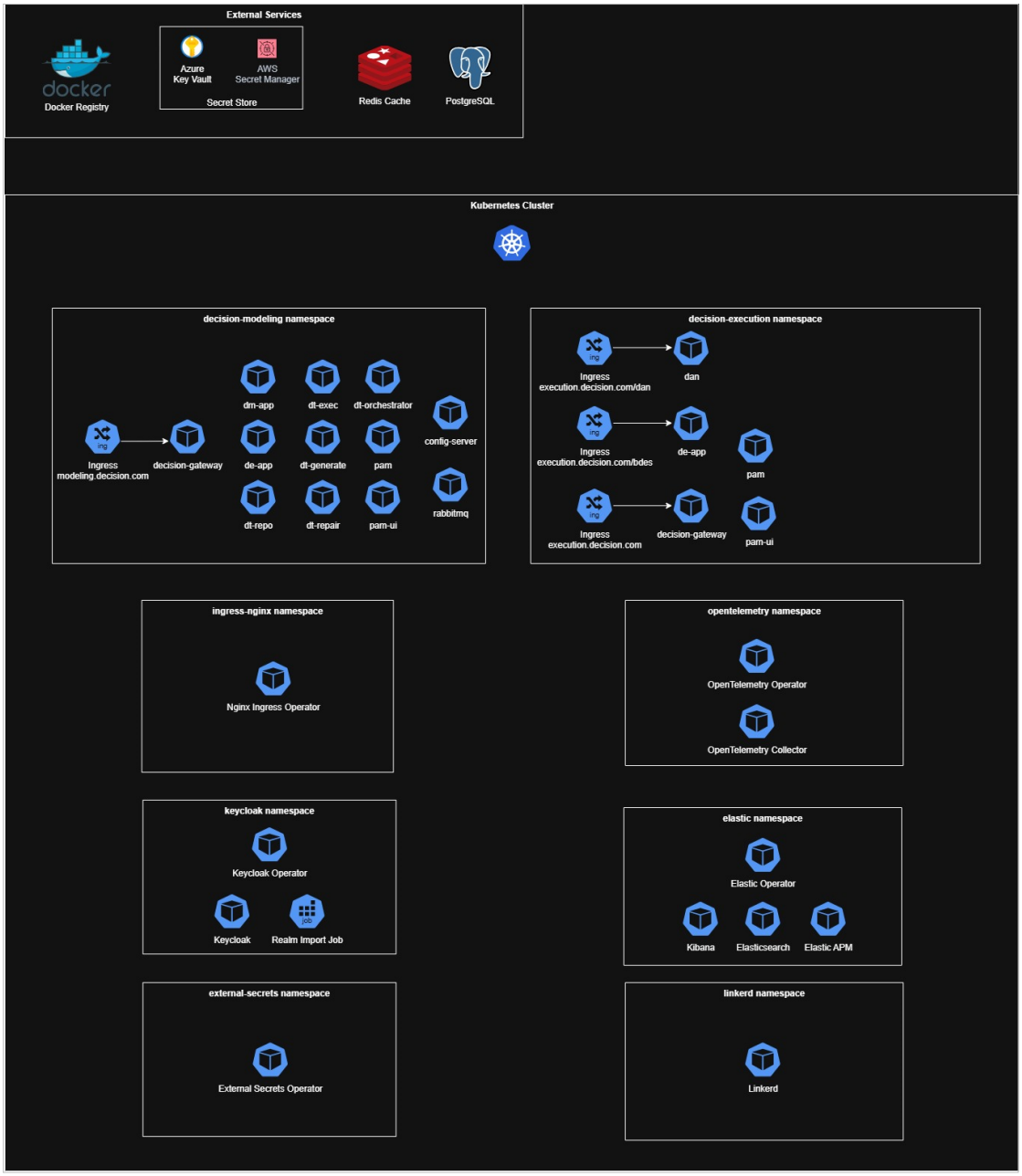
Containerized deployment is an advanced variant of standalone deployment. In this model, Sapiens provides application components as JAR files embedded within Docker images, ready to be deployed in environments such as:

- Kubernetes
- OpenShift
- Azure Kubernetes Service (AKS)
- Amazon Elastic Kubernetes Service (EKS)

For complete deployment steps, refer to the Helm chart's README.md file, as outlined in the Accessing the README.md File for Deployment Instructions section.

Advantages of Containerized Deployment:

- **Pre-Packaged Application JARs**: Application JARs are bundled within Docker images, simplifying deployment.
- **Isolated Failure Points**: A failure in one application component does not affect others, except when dependencies exist.
- **Flexibility and Scalability**: Components can be independently scaled and managed.
- **Fast Startup**: Each application component can be rapidly started in isolation.
- **Resource Optimization**: Independent memory and resource allocation is provided for each component, tailored to its requirements.
- **Enhanced Monitoring**: Each application runs as an independent Java process, enabling granular monitoring.
- **Fine-Grained Scaling**: Individual components can be scaled as needed.



5.2.2 Accessing the README.md File for Deployment Instructions

Follow the steps below to locate the Helm chart's README.md file:

- Locate the downloaded package file.
- Extract the package using any archive tool (e.g., WinRAR, 7-Zip).
- Navigate to the dan directory in the extracted folder.
- Open the README.md file available at the root of the dan folder.

```
total 64
-rw-r--r--   1    9230  1 Jul 19:47 values.yaml
-rw-r--r--   1   11484  1 Jul 19:47 README.md
-rw-r--r--   1     241  1 Jul 19:47 Chart.yaml
-rw-r--r--   1     239  1 Jul 19:47 Chart.lock
drwxr-xr-x  13     416 17 Jul 18:19 templates
drwxr-xr-x   3      96 17 Jul 18:19 charts
INLT-G7WMJLYK42-$
```

5.2.3 Package Content

The containerized deployment package includes the following essential components:

- Docker images and associated files for each application component.
- Helm charts for managing deployment in Kubernetes environments.

5.2.4 Recommended Use Cases

Containerized deployment is recommended in the following scenarios:

- Deployment to a containerized environment (e.g., Kubernetes, OpenShift, AKS, EKS) is required.
- A preference exists for pre-packaged Docker images for simplified deployment.
- Requirement for scalability, resource allocation, monitoring, and isolation at the per-application level.
- Environments requiring fast recovery and startup of each application component.
- High-availability production environments with a large number of users and/or high request rates.

5.2.5 Prerequisites

Ensure the following platform requirements are fulfilled before initiating containerized deployment:

- **Compatible Containerized Environment**: Kubernetes, OpenShift, AKS, or EKS.
- **DAN Requirements**:
  - Docker Image
  - Database schema granted full read/write permissions on all schema objects (e.g., tables, sequences, etc.). Refer to the Database Implementation section for database-specific instructions.
- **DE Requirements**:
  - Docker Image
  - Database schema granted full read/write permissions on all schema objects (e.g., tables, sequences, etc.). Refer to the Database Implementation section for database-specific instructions.
- **Decision Gateway**:

- Docker Image
- Redis Database.

- **Configuration Service (Optional)**:
  - **Rabbit MQ**:
    - **Docker Image for RabbitMQ**: This is a pre-built, standardized package that allows you to run RabbitMQ easily using Docker.
    - **Tag: 3-management**: This indicates a specific version of the RabbitMQ Docker image, with the "management" plugin enabled. The management plugin provides a user-friendly web-based management UI for RabbitMQ.
- **OAuth IDP (e.g., Keycloak)**: Required for user authentication using OAuth 2.0.
- **Ingress NGINX Controller**: Used for managing external access.
- **APM Tool (Optional)**: Application Performance Monitoring (e.g., Elastic ECK).
- **External Secrets Operator (Optional)**
- **Open Telemetry Configuration (Optional)**:

  OpenTelemetry is an open-source observability framework for generating, collecting, processing, and exporting telemetry data (such as traces, metrics, and logs) from applications.

  Components of OpenTelemetry Configuration:
  - OpenTelemetry Operator
  - OpenTelemetry Collector
  - Auto-Instrumentation Configuration

5.2.6 Installation

For detailed installation instructions, refer to the Helm chart's README.md file, as explained in the Accessing the README.md File for Deployment Instructions section.

5.3 All-in-One Deployment

The all-in-one deployment model is a traditional approach where all Decision components are deployed on a single host machine, operating under one application server (such as Tomcat). This approach simplifies management but has limitations in terms of scalability and resource optimization.

5.3.1 Overview

The following are brief descriptions of prerequisite directories included in the current DAN release and mentioned throughout this document.

Folder	Description
Conf	Configuration files for DAN configuration

Folder	Description
War	App WAR file to be installed on the Tomcat Application Server
Release Notes	The Release Notes document

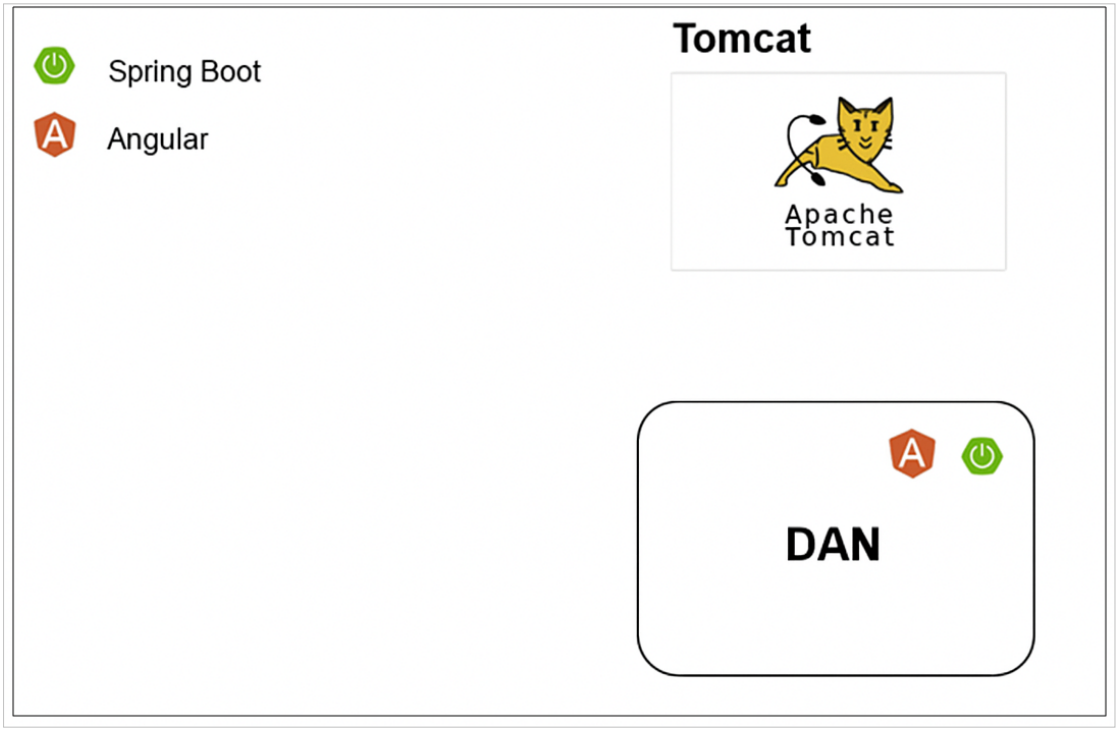
Note: Once the installation is complete and Tomcat is started, a DAN instance will be running.

Advantages:

- Centralized configuration and maintenance for all Decision components.
- No network latency between components, as they run on the same host.

Disadvantages:

- Single point of failure – an issue with one component can impact the entire application.
- Lack of fine-grained resource allocation for individual components.
- Larger application archive and memory footprint.
- Slower scaling, as all components must be scaled together.



All-in-One deployment Architecture

5.3.2 Package Content

This section outlines the components included in the all-in-one deployment package:

- **Web Application Archive (WAR)**: Application components are packaged as WAR files, designed to be deployed on a single application server (e.g., Apache Tomcat).

5.3.3 Recommended Use Cases

The all-in-one deployment method is recommended in the following scenarios:

- The customer prefers to host the Decision application on an existing web application server.
- When centralized configuration through a single application server, unified deployment location, and single-port access is prioritized over:
    - Per-application instance scalability
    - Dedicated resource allocation
    - Isolated monitoring
    - Component-level isolation

5.3.4 Prerequisites

This section outlines the essential system and platform requirements for deploying Decision using the war deployment model. Ensure all the following prerequisites are met before starting the installation.

Platform Requirements:

- **Application Server:**
    - Tomcat Application Server (version 10 or above)
- **Java Development Kit (JDK):**
    - Installed JDK distribution of Java 17
    - The JDK installation path should be set in the **JAVA_HOME** environment variable, either by the operating system or directly within the Tomcat application server.
- **Disk Space:**
    - Minimum of **2GB** of available disk space (including the Tomcat server).
- **Memory (RAM):**
    - For basic usage (application startup): At least 3GB of RAM.
    - For optimal performance in most production environments: At least 8GB of RAM

Database Requirements:

- **Database Server:**
    - PostgreSQL (refer to each application installation guide for minimum supported versions and database-specific instructions).
- **Dedicated Database Schema:**
    - Each Decision application must have a dedicated schema with full read/write permissions on all schema objects (tables, sequences, etc.).
    - Multiple schemas can exist under the same database server instance, but each must be isolated per application.

Application Package (WAR Files):

The following web application archive (WAR) files are required for a complete WAR deployment:

- Mandatory Components:
    - **dan.war** - Decision Analytics (DAN) Application

5.3.5 Installation

This section provides a comprehensive, step-by-step guide to install the Decision components in a WAR deployment using the Apache Tomcat application server. Ensure that all prerequisites (Java, Tomcat, WAR files, and database setup) are fulfilled before proceeding.

Step 1: Set Up Global Classpath and Configuration Folders

1. **Edit Tomcat Global Configuration**
    - Navigate to the Tomcat configuration directory:
      <Tomcat folder>/conf/catalina.properties
    - Locate the shared.loader property and append the Decision configuration directory path:

      ```
      shared.loader=${catalina.home}/decision-config
      ```

    - This allows Tomcat to load shared configurations across all deployed WAR applications.
2. **Create the Configuration Directory**
    - If it does not already exist, create a directory named **decision-config** in the Tomcat root directory.
3. **Create Configuration Sub-Directories per Application**
    - Inside the decision-config directory, create the following sub-directory for DAN application:
        - dan-config
4. **Add Configuration Files**
    - Within the application's subdirectory, add your application configuration file:
        - Either **application.properties** or **application.yml**, depending on your format preference.

Step 2: Configure Application Contexts in Tomcat

1. **Edit the server.xml File**
    - Open the following file: <Tomcat folder>/conf/server.xml
    - Inside the <Host> tag, add a <Context> entry for each Decision application. For example:

```
<Host
name
="localhost"

appBase
="webapps"
 unpackWARs="true" autoDeploy="true" startStopThreads="10">
    <Context docBase="dan" path="/dan" />
    <Context docBase="dm" path="/dm" />
    <Context docBase="dt" path="/dt" />
    <Context docBase="bdes" path="/bdes" />
</Host>
```

- The docBase should match the WAR file name (without the .war extension).
- The path defines the application's context URL.

2. **Adjust Start-Stop Threads:**
   - Set the startStopThreads attribute based on the number of <Context> entries.
   - The recommended value is equal to the total number of applications being deployed.

Step 3: Configure HTTP and Shutdown Ports

1. **Set HTTP Port:**
   - Still in server.xml, modify the HTTP <Connector> tag as needed. For example, to use port 9080, update the configuration accordingly. The default port is 8080.

```
<Connector port="9080" protocol="HTTP/1.1"
           connectionTimeout="20000"
           redirectPort="8443"
           maxParameterCount="1000" />
```

2. **Set Shutdown Port**
   - Change the shutdown port to avoid conflicts with other local servers. The default shutdown port is 8005.

```
<Server port="9005" shutdown="SHUTDOWN">
```

   Avoid using the default 8005 if multiple Tomcat instances are running on the same host.

Step 4: Create and Configure setenv.sh Script for Tomcat

1. **Create the setenv.sh Script**
   - If not already present, create a setenv.sh file in the <Tomcat folder>/bin directory.
2. **Add Environment Variables and JVM Options**

   Edit **setenv.sh** and add the following content:

```
#!/bin/sh

export JAVA_HOME="/home/.jdks/corretto-17.0.14"

export JAVA_OPTS="-Xmx8g \
-Ddecision.home=${catalina.home} \
-Ddecision.dan.home=${catalina.home} \
-Dsun.io.useCanonCaches=true \
-XX:+UseParallelGC \
-Dfile.encoding=UTF-8 \
-Dcom.sun.net.ssl.enableECC=true \
--add-opens=java.base/sun.reflect.annotation=ALL-UNNAMED \
--add-modules=java.se \
--add-exports=java.base/jdk.internal.ref=ALL-UNNAMED \
--add-opens=java.base/sun.nio.ch=ALL-UNNAMED \
--add-opens=java.base/java.lang=ALL-UNNAMED \
--add-opens=java.management/sun.management=ALL-UNNAMED \
--add-opens=jdk.management/com.sun.management.internal=ALL-UNNAMED \
--add-opens=java.base/java.util=ALL-UNNAMED"
```

3. **Edit Configuration Based on Your Environment**
    - Set **JAVA_HOME** to the absolute path of your installed Java 17 JDK.
    - Modify the **-Xmx8g** flag to allocate appropriate maximum heap memory depending on system resources.

Step 5: Prepare Database Schemas

Before deploying the applications, ensure the database environment is ready.

- Verify that database schemas for each Decision application (DAN, DE, DM and DT) are created.
- Each schema should have sufficient read/write privileges, as defined in the Prerequisites section of this guide.

Step 6: Deploy Application WAR Files

1. **Stop Running Tomcat Server**
    - If any Decision applications are running on the target Tomcat server, stop the server.
2. **Copy WAR Files**
    - Copy the Decision application WAR files to the **<Tomcat folder>/webapps** directory.
3. **Rename WAR Files**
    - Remove the version numbers from the WAR filenames:

      e.g., dan-1.0.war → dan.war
4. **Clean Up Existing Files**
    - Delete any existing WAR files or folders of a Decision application you are about to redeploy.

Step 7: Configure Application Properties

- Refer to the DAN Configuration section of this document.
- Populate the configuration file with all required properties under the section Customized Configuration Properties.

5.3.6 DAN Server – Starting and Stopping

Note: Before starting the server, you can validate that all configurations are correct by running the version.sh script.

Example: {TOMCAT_HOME}/bin/version.sh

To Start the DAN Server:

- Run:

```
{TOMCAT_HOME}/bin/startup.sh
```

To Stop the DAN Server:

- Run

```
{TOMCAT_HOME}/bin/shutdown.sh
```

Note: To verify that the server is running without any errors, use the following curl command: (Assuming the default context path /dan)

Example:

```
curl http://<host>:<port>/dan/actuator/health
```

If the connection fails, possible causes include:

- Incorrect port, IP address, or DNS
- Firewall or security group blocking access

Check the following logs for more details:

- DAN logs: dan/log
- Tomcat logs: {TOMCAT_HOME}/logs

5.3.7 Upgrade Procedure

To upgrade DAN to the current version, perform the following steps:

1. Stop the application server.
2. Back up your data, database, and/or file system repository.
3. Back up the **dan** directory and **dan.war** file located in **{tomcat_home}/webapps**.
4. Delete the **dan** directory and **dan** file from **{tomcat_home}/webapps**.

5. Copy the new **dan.war** file from the release package location to **{tomcat_home}/webapps**.
6. Ensure the configuration file **{dan_home}/conf/dan-config/application.properties** is updated as needed.
7. Start the upgraded DAN server and review the DAN logs to ensure there are no errors. Schema changes for the current version will be applied automatically, and an audit of these changes can be found in the DAN logs or in the DAN database table **flyway_schema_history**.
8. Once the DAN application is running, perform a connection test using a web browser or curl.

```
curl http://<host>:<port>/dan/actuator/health
```

5.4 Standalone Deployment

5.4.1 Overview

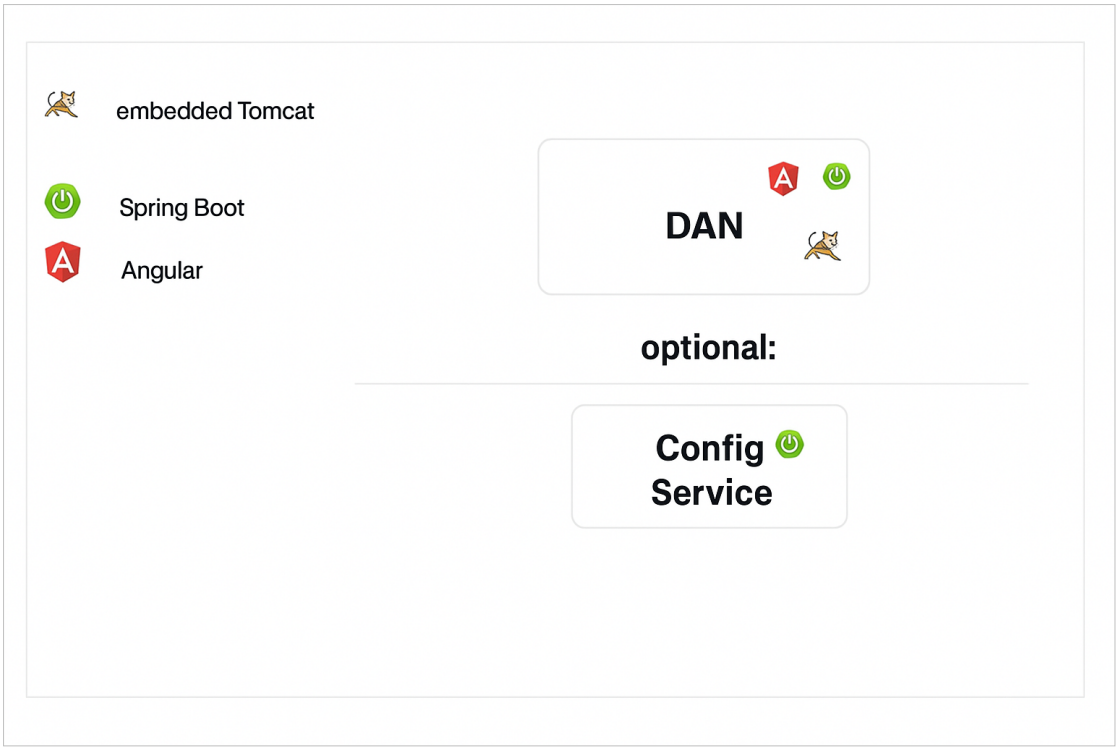
In standalone deployment, each Decision application is packaged as a self-executable JAR file generated using the Spring Boot framework. These JAR files contain an embedded Tomcat server, making them completely independent and self-sufficient.

Key Characteristics:

- **Independent Execution:**
  - Each application runs on its own port and can be hosted on the same or different machines.
- **Application-Specific Configuration:**
  - Server-related properties (e.g., port, SSL) are managed directly in the application's configuration file (application.properties or application.yml) or via Java command-line arguments.
- **Improved Flexibility and Scalability:**
  - This model allows fine-grained control over resource allocation, monitoring, and scaling for each application.

Advantages:

- Distributed points of failure — a failure in one component does not impact others.
- Maximum flexibility, scalability, and isolation between application components.
- Fast startup due to isolated application instances.
- Independent memory and resource allocation for each component.
- Per-application monitoring using separate Java processes.
- Fine-grained scaling for each application.



DAN Standalone Deployment Architecture

5.4.2 Package Content

The standalone deployment package includes the following components:

- **Application Java Binary Files:**

  Compiled Java class files for each Decision application.
- **Spring Boot Framework Files:**

  Essential Spring Boot classes required for application startup and management.
- **Java Third-Party Libraries:**

  External JAR files used by the application for various dependencies.
- **Maven Build Metadata:**

  Configuration details, including project structure, dependencies, and build information.

5.4.3 Recommended Use Cases

Standalone deployment is ideal in the following scenarios:

- When you prefer to build your own container images using the application JAR files.
- When fine-grained scalability, resource allocation, monitoring, and isolation per application are required.
- When fast recovery and startup times for each application are critical.

Common Use Cases:

- Production environments with a large number of users.
- High-traffic scenarios involving frequent server requests.

5.4.4 Prerequisites

The standalone deployment of Decision requires specific platform dependencies for each component to ensure optimal performance and smooth operation. These prerequisites include Java installation, available disk space, memory allocation, database setup, and connectivity between components.

Java Installation (Global Requirement)

- **JDK Version:**

  Installed JDK distribution of Java 17.

- **JAVA_HOME Configuration:**

  Ensure that the Java installation path is correctly set in the JAVA_HOME environment variable, either at the operating system level or through the application's Java command line.

- **Memory Allocation (-Xmx):**

  The memory allocation size for each application should be determined based on the expected user load and the number of concurrent requests. This can be configured via Java command-line arguments.

  Example:

  ```
  java -Xmx1g -jar <app_jar_file_path>
  ```

- **Disk Space (Minimum Requirements)**

  Each application has specific disk space requirements, which are listed below (list to be added as per actual application specs).

Component	Disk Space	Memory (Xmx)	Additional Dependencies
DAN	185 MB	Minimum: 8 GB	Dedicated database schema (PostgreSQL).

5.4.5 Installation

To install any Decision application in standalone mode, follow the steps below:

1. **Database Schema Setup**
   - If the application requires a database schema, ensure it already exists.
   - If the schema does not exist, create it before proceeding.
2. **Application JAR Placement**
   - Copy the self-executable JAR file of the application to your desired application home folder.
3. **Configuration File Setup**
   - Create or copy the appropriate configuration file (application.properties or application.yml).
   - Ensure this file is placed in the same folder as the application JAR file.

4. **Configuration Settings**
    - Refer to the DAN Configuration section of this document.
    - Populate the configuration file with all required properties under the section Customized Configuration Properties.

5.5 Post Installation Validation

5.5.1 Health Check Validation

The following endpoint is available to perform heartbeat and health checks for the DAN application:

- https://<host>:<port>/dan/actuator/health

  Example: http://localhost:9081/dan/actuator/health

5.5.2 DAN Startup Process and Logs

Depending on the deployment setup, the DAN application will start as part of the Tomcat initialization process.

Once the DAN application starts, log files will be created at the location specified by the **logging.file.path** variable. Additional log files can also be found in the **{TOMCAT_HOME}/logs** directory.

The primary DAN log file is named **dan.log** and contains most runtime information. Some additional logs may be stored in other files depending on configuration.

Sample DAN log entries at startup (actual messages may vary):

- **INFO [ServletWebServerApplicationContext] Root WebApplicationContext**: initialization started
- **INFO [c.z.h.HikariDataSource] [] HikariPool-1 - Starting...** (Indicates database connection initialization)
- **INFO [ServletWebServerApplicationContext] Root WebApplicationContext**: initialization completed in xxx ms (Marks completion of the startup process)

6. DAN Database Implementation (Optional)

The following implementation guide is for unencrypted access. For encrypted access, refer to the Property Value Encryption section.

6.1 PostgreSQL

Follow these steps to configure PostgreSQL for the DAN application:

1. **Edit application.properties**

   Update or create the application's application.properties file

2. **Define or Use an Existing Database User**

   You can either create a new user or use an existing one (e.g., DanPostgresUser).

3. **Update Configuration Placeholders**

   Replace the following placeholders in the application.properties file:

   - spring.datasource.url – Specify the JDBC URL of the database
   - spring.datasource.username – Specify the database username
   - spring.datasource.password – Specify the database user's password.

   Example Configuration:

   ```
   spring.datasource.url=jdbc:postgresql://localhost:5432/
   {databaseName}?currentSchema=dan
   spring.datasource.username=DanPostgresUser
   spring.datasource.password=DanPostgresPass
   ```

4. **Create the Database**

   Create a PostgreSQL database if it does not already exist.

5. **Create the DAN Schema**

   Within the PostgreSQL database, it is mandatory to create the DAN application schema under default postgres database for the DAN application.

7. DAN Configuration

Configuration Properties in Decision Analytics Server are settings that determine how the system behaves. This section describes the customizable properties that system administrators can use to control the behavior of Decision Analytics Server components. Only authorized personnel should modify the contents of the property files.

7.1 Multi-Thread Pool Configuration

The multi-thread configurations define thread pools for concurrent task execution in your Spring Boot application. Improves application throughput and responsiveness by enabling parallel processing of tasks, reducing bottlenecks and optimizing resource usage

7.1.1 Message Thread Pool Configuration

These properties configure the thread pool for message processing, including core and maximum thread counts and queue capacity, to optimize concurrency and resource usage.

Name	Valid Values	Default Value	Description
decision.dan.threadpool.message.core-size	Integer	25	Specifies the minimum number of threads always kept alive for message processing.
decision.dan.threadpool.message.max-size	Integer	50	Defines the maximum number of threads allowed in the message thread pool.
decision.dan.threadpool.message.queue-capacity	Integer	10	Sets the number of tasks that can wait in the queue before new threads are created.

7.1.2 Async Database Thread Pool Configuration

These properties configure the thread pool for asynchronous database operations, including core and maximum thread counts and queue capacity, to optimize concurrent DB task execution.

Name	Valid Values	Default Value	Description
decision.dan.threadpool.db.core-size	Integer	20	Specifies the minimum number of threads always kept alive for database tasks.
decision.dan.threadpool.db.max-size	Integer	20	Defines the maximum number of threads allowed in the database thread pool.

Name	Valid Values	Default Value	Description
decision.dan.threadpool.db.queue-capacity	Integer	200	Sets the number of database tasks that can wait in the queue before new threads are created.

7.2 Health Check and Heartbeat Configuration

These properties control the health check endpoints and heartbeat URLs used to monitor the application's status. They define the endpoints where heartbeat requests are sent, configure timeouts, and specify which health details and probes are exposed through Spring Boot Actuator. System administrators can disable certain checks (such as Database or SSL) to customize what is reported, based on operational requirements.

Name	Valid Values	Default Value	Description
decision.dm.heartbeat-url	String (path)	/ws/rs/heartbeat	Defines the endpoint used for DM service heartbeat checks.
decision.dt.heartbeat-url	String (path)	/heartbeat	Defines the endpoint used for DT service heartbeat checks.
decision.dt.orch.base-url	String (path)	http://localhost:8085/dt-orchestrator	Specifies the base URL of the DT orchestrator service.
decision.health.heartbeat.timeout	Integer	3000	Sets the timeout (in milliseconds) for heartbeat responses.
management.endpoint.health.probes.enabled	True/False	true	Enables or disables health probes.
management.endpoint.health.show-details	String	always	Controls the level of health details displayed.
management.endpoints.web.exposure.include	String	health,info	Specifies which Actuator endpoints are exposed.
management.health.db.enabled	True/False	false	Enables or disables the database health check.
management.health.defaults.enabled	True/False	false	Enables or disables default health checks.
management.health.ssl.enabled	True/False	false	Enables or disables the SSL health check.

7.3 DataSource Configuration

These properties define how the application connects to and manages the database, including pool-specific overrides, Hibernate settings, Flyway migrations, transaction persistence, and performance optimizations to ensure efficient and reliable data operations.

Name	Valid Values	Default Value	Description
decision.dan.db.tuning.enable-nest-loop	ON/OFF	OFF	Controls whether nested loop joins are used in query execution. When ON, nested loop joins are allowed; when OFF, the database avoids nested loops and prefers alternative join strategies.
decision.dan.db.tuning.jit	ON/OFF	OFF	Controls the use of Just-In-Time (JIT) compilation for query execution. When ON, JIT optimization is enabled to potentially improve performance; when OFF, JIT is disabled.
decision.dan.db.tuning.maintenance-work-mem	Memory Size	512MB	Assigns memory for database maintenance tasks to speed up operations like indexing.
decision.dan.db.tuning.synchronous-commit	ON/OFF	OFF	Controls transaction commit behavior. When ON, transactions wait for confirmation that data is safely written to disk; when OFF, commits return faster with reduced durability guarantees.
decision.dan.db.tuning.work-mem	Memory Size	512MB	Allocates memory for query processing, enhancing performance for complex operations.
decision.dan.persist.de.transactions	true/false	true	Controls whether DAN transaction data is persisted. When true, transaction data is stored; when false, transaction data persistence is disabled.
spring.jpa.show-sql	true/false	false	Controls SQL logging in application logs. When true, executed SQL statements are logged; when false, SQL logging is disabled.

7.4 Batch Configuration

These properties schedule and control periodic statistics calculations and transaction purging, helping automate data maintenance and cleanup in the application.

Name	Valid Values	Default Value	Description
decision.dan.execution.statistics.scheduler.job.cron	Quartz cron expression	0 0 0 * * ?	Defines the schedule for the statistics calculation job (daily at 00:00 AM).
decision.dan.execution.statistics.scheduler.job.preceding.days	Integer	30	Specifies the number of preceding days of data to include in statistics calculations.
decision.dan.purge.transaction.batch.cron	Quartz cron expression	0 0 3 * * ?	Defines the schedule for the transaction purge job (daily at 3:00 AM).
decision.dan.purge.transaction.retentionDays	Integer	90	Sets the number of days transactions are retained before purging.

7.5 Quarterly Threshold Configuration

This section configures the licensing tier thresholds used for quarterly billing calculations and usage forecasting. The system automatically divides these annual values by 4 to determine quarterly limits and calculate overage charges based on actual decision and fact processing volumes.

Name	Valid Values	Default Value	Description
decision.dan.quarterly.decision.l1	Positive integer	500000000	Level 1 threshold for quarterly decisions (125M per quarter).
decision.dan.quarterly.decision.l2	Positive integer greater than L1	1000000000	Level 2 threshold for quarterly decisions (250M per quarter)
decision.dan.quarterly.fact.l1	Positive integer	5000000000	Level 1 threshold for quarterly facts (1.25B per quarter)
decision.dan.quarterly.fact.l2	Positive integer greater than L1	10000000000	Level 2 threshold for quarterly facts (2.5B per quarter)

7.6 OpenAI Model Configuration

These properties configure how your application connects to an OpenAI model endpoint, including API location, authentication, version, and request timeouts.

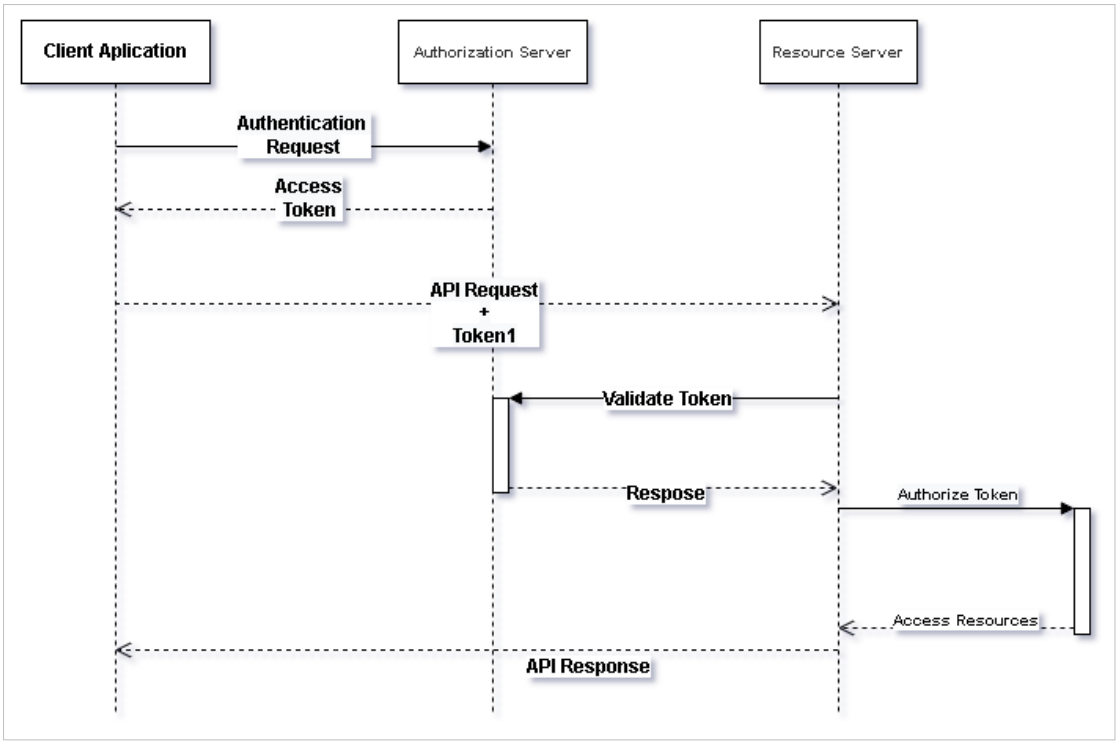
Name	Valid Values	Default Value	Description
decision.openai.client.api-key	String	none	Defines the API key used for authenticating requests to the OpenAI service.
decision.openai.client.api-version	String	none	Specifies the version of the OpenAI API to be used.
decision.openai.client.base-uri	String (URL)	none	Specifies the base URI of the OpenAI model endpoint.
decision.openai.client.timeout.connect	Integer	30000	Sets the connection timeout (in milliseconds) for requests to the OpenAI API.
decision.openai.client.timeout.receive	Integer	30000	Sets the receive timeout (in milliseconds) for responses from the OpenAI API.

7.7 Security Configuration

7.7.1 OAuth 2.0 Authentication

OAuth 2.0 is a standard designed for online authorization. It allows a website or application to access resources hosted by other web apps on behalf of a user, without exposing the user's credentials.

To integrate OAuth 2.0 with DAN, complete the following configuration:



OAuth 2.0 Configuration

To configure DAN for OAuth 2.0 authentication, use the following properties in the application.properties file:

```
# Activate OAuth2 profile
spring.profiles.active=oauth2

# JWT audience validation (expected 'aud' claim in token)
spring.security.oauth2.resourceserver.jwt.audiences=https://dan-for-
testing

# Identity Provider configuration
spring.security.oauth2.resourceserver.jwt.issuer-uri=<IDP issuer URL>
spring.security.oauth2.resourceserver.jwt.jwk-set-uri=<IDP jwk set-uri
URL>
```

Note: The jwk-set-uri property is optional but recommended for enhanced security.

7.7.2 Basic Authentication

The basic authentication is managed within your Spring Boot application using credentials defined in **application.properties**. This allows you to:

- Secure APIs and resources from unauthorized access.
- Control user roles and permissions without relying on external identity providers.
- Enable basic authentication for development, testing, or simple deployments.

This ensures that only users with valid credentials can access protected endpoints.

Example Configuration (application.properties)

```
# Define internal user
decision.dan.security.users[0].username=user1
decision.dan.security.users[0].password=pass1

# Assign roles to the user (controls access levels)
decision.dan.security.users[0].roles=VIEWER,GUEST
```

7.7.2.1 DAN Permissions

DAN permissions are specific actions granted to a user. They are configured under:

```
decision.dan.security.users[0].permissions
```

Each permission typically maps to a feature, module, or API endpoint within the application. This allows fine-grained control beyond just roles.

Example Configuration (application.properties)

```
# Define internal user
decision.dan.security.users[0].username=user1
decision.dan.security.users[0].password=pass1
decision.dan.security.users[0].roles=VIEWER,ADMIN

# Assign DAN permissions to the user
decision.dan.security.users[0].permissions=DAN_AI_READ,DAN_DOWNLOAD_
TRANSACTION
```

The following table lists the available DAN permission groups and their corresponding descriptions, defining the specific actions users are allowed to perform within the application.

Permission Group	Description
DAN_AI_READ	Permission to view the AI dashboard.
DAN_DECISION_CONCLUSION_ DASHBOARD_READ	Permission to view the decision conclusion dashboard.
DAN_TRANSACTION_DASHBOARD_READ	Permission to view the transaction dashboard.
DAN_EXECUTION_DASHBOARD_READ	Permission to view the execution dashboard.
DAN_PERFORMANCE_DASHBOARD_READ	Permission to view the performance dashboard.
DAN_SEND_TRANSACTION_TO_DM	Permission to send transactions to DM.
DAN_DOWNLOAD_TRANSACTION	Permission to download transactions.

7.7.3 Internal Authentication

The internal authentication mode allows the DAN application to communicate with DM to fetch a specific DM user for authentication and authorization.

Refer to the **Decision Analytics Permissions** section in the *Decision Analytics (DAN) User Guide* for details about available roles and permissions to configure the user appropriately.

Note: Internal authentication mode requires OAuth client credentials to be defined for the target DM, where the OAuth machine user is available.

7.8 OAuth2 Client Registration Properties

Registrations are used to configure OAuth2 client details for an application. Each registration defines a client application that interacts with an OAuth2 provider. System administrators can define multiple registrations to support different providers or use cases. The table below shows the configuration properties:

Name	Valid Values	Default Value	Description
spring.security.oauth2.client.registration.<registration-id>	String (e.g., "dan")	None (must be provided)	A unique name for the client registration. Replace <registration-id> with the actual identifier.
spring.security.oauth2.client.registration.<registration-id>.client-id	String provided by OAuth2 provider	None (must be provided)	The unique identifier for the client application registered with the OAuth2 provider. It is used to identify the application during authentication.
spring.security.oauth2.client.registration.<registration-id>.client-secret	String provided by OAuth2 provider	None (must be provided)	The secret key associated with the client application, used to authenticate the client with the OAuth2 provider.
spring.security.oauth2.client.registration.<registration-id>.scope	Comma-separated list of scopes (e.g., "DAN_USER, DAN_ADMIN, DAN_MONITORING")	None (must be provided)	Defines the permissions or access levels that the client application is requesting from the OAuth2 provider.
spring.security.oauth2.client.registration.<registration-id>.authorization-grant-type	authorization_code, client_credentials, password, refresh_token	None (must be provided)	Specifies the OAuth2 grant type (flow) that the client application will use to obtain an access token.

Name	Valid Values	Default Value	Description
spring.security.oauth2.client.registration.<registration-id>.provider	Reference to a provider defined under spring.security.oauth2.client.provider (e.g., "dan-provider")	None (must be provided)	Associates the client registration with a specific OAuth2 provider configuration.
spring.security.oauth2.client.provider.<provider-id>.token-uri	Valid URL provided by OAuth2 provider	None (must be provided)	The endpoint URL where the client application can request an access token from the OAuth2 provider.

7.9 DM Configuration for Importing Test Cases

These properties configure transaction export limits and define the endpoint for connecting to the Decision Manager (DM) service, specifically used during test case import operations.

Name	Valid Values	Default Value	Description
decision.dan.execution.transactions.export.max-items	Integer	10000	Defines the maximum number of transactions that can be exported in a single operation.
decision.dm.base-url	String (path/URL)	http://localhost:9080/dm	Specifies the base URL for the Decision Manager (DM) service. This URL is used by the application when importing test cases.

7.10 Jasypt Configuration

These properties configure Jasypt for decrypting sensitive values in your configuration files, specifying the decryptor bean and how encrypted properties are identified.

Name	Valid Values	Default Value	Description
jasypt.encryptor.bean	String	jasyptDecryptBean	Name of the Jasypt encryptor bean used for decrypting encrypted configuration properties.
jasypt.encryptor.property.prefix	String	ENC:	Prefix used to identify encrypted properties that require decryption.

Name	Valid Values	Default Value	Description
jasypt.encryptor.property.suffix	String	Not set	Optional suffix used to identify encrypted properties for decryption.

7.11 Environment and Server Configuration

These properties define which profiles are active, the application's context path, and the port it runs on.

Name	Valid Values	Default Value	Description
server.port	Integer	8080	Specifies the port number on which the embedded server listens for HTTP requests.
server.servlet.context-path	String (path)	/dan	Defines the root context path for the application’s web endpoints.
spring.profiles.active	String	basic	Specifies the active Spring profiles (basic, internal) for environment-specific configuration. The basic profile allows the application to use the permissions defined in the configuration for each user. Refer to the **Basic Authentication** section under Security Configuration to configure users with the available permissions.

7.12 Kafka Configuration Properties

Kafka properties in DAN are required to enable efficient, scalable, and reliable communication between microservices for processing, transferring, and consuming large volumes of real-time data. They configure how DAN connects to Kafka brokers, manages message production and consumption, ensures data integrity, and supports concurrency and fault tolerance in distributed environments.

Property	Valid Values	Default Value	Description
spring.kafka.admin.properties.sasl.jaas.config	JAAS config string	Not set	JAAS configuration for SASL authentication in admin client.

Property	Valid Values	Default Value	Description
spring.kafka.admin.properties.sasl.mechanism	PLAIN, SCRAM-SHA-256, etc.	Not set	SASL mechanism for admin client authentication.
spring.kafka.admin.properties.security.protocol	PLAINTEXT, SSL, SASL_PLAINTEXT, SASL_SSL	PLAINTEXT	Security protocol for Kafka admin operations.
spring.kafka.bootstrap-servers	String (<host>:<port>)	localhost:29092	Specifies the Kafka broker addresses used to connect producers and consumers. This property is always required.
spring.kafka.consumer.auto-commit-interval	Integer (time in ms)	100ms	Interval for auto-committing offsets when enable-auto-commit is set to true.
spring.kafka.consumer.auto-offset-reset	earliest, latest, none	earliest	Determines the behavior when no offset is found. earliest starts from the beginning of the topic.
spring.kafka.consumer.client-id	String	de-client	Assigns a unique client ID to the DE consumer for logging and tracking.

Property	Valid Values	Default Value	Description
spring.kafka. consumer. enable-auto- commit	True / False	TRUE	Enables automatic offset commits for consumers. Useful for simple offset management, but may reduce reliability.
spring.kafka. consumer. group-id	String	de-client-grp	Specifies the consumer group ID for DE. Used for load balancing and fault tolerance.
spring.kafka. consumer. key-deserializer	Fully qualified class name	org.apache. kafka. common. serialization. StringDeseriali zer	Deserializer class for consumer message keys.
spring.kafka. consumer. max-poll-records	Integer	1000	Maximum number of records returned in a single poll. Controls batch size.
spring.kafka. consumer. properties.acks	all, 1, 0	all	Number of acknowledgme nts required from Kafka before considering a request successful (rarely used for consumers).
spring.kafka. consumer. properties. confluence. cred.source	USER_INFO, SASL_INHERIT	Not set	Credential source for Confluent Schema Registry.

Property	Valid Values	Default Value	Description
spring.kafka.consumer.properties.enable-idempotence	True / False	TRUE	Enables idempotence for exactly-once message processing (rarely used for consumers).
spring.kafka.consumer.properties.max-in-flight-requests-per-connection	Integer	1	Maximum number of unacknowledged requests allowed per connection. Helps ensure ordering with idempotence.
spring.kafka.consumer.properties.sasl.jaas.config	JAAS config string	Not set	JAAS configuration string for SASL authentication when security protocol is SASL.
spring.kafka.consumer.properties.sasl.mechanism	PLAIN, SCRAM-SHA-256, etc.	Not set	SASL mechanism used for authentication.
spring.kafka.consumer.properties.schema.registry.url	URL	Not set	Schema registry URL for Avro/Protobuf serialization.
spring.kafka.consumer.properties.schema.registry.user.info	username:password	Not set	Schema registry credentials for secured registry access.
spring.kafka.consumer.properties.ssl.endpoint.identification.algorithm	https, (empty string to disable)	https	SSL endpoint identification algorithm. Use https for hostname verification.

Property	Valid Values	Default Value	Description
spring.kafka. consumer. properties. tenantIds	Comma-separated tenant IDs	Not set	Defines tenant IDs for multi-tenancy scenarios.
spring.kafka. consumer. security. protocol	PLAINTEXT, SSL, SASL_PLAINTEXT, SASL_SSL	PLAINTEXT	Defines the security protocol for consumer connections.
spring.kafka. consumer. topic	String	example-topic	Defines the topic used for DE message consumption.
spring.kafka. consumer. value-deserializer	Fully qualified class name	org.apache. kafka. common. serialization. StringDeseriali zer	Deserializer class for consumer message values.
spring.kafka.dm. metadata. auto-offset-reset	earliest, latest, none	earliest	Determines the behavior when no offset is found. earliest starts from the beginning of the topic.
spring.kafka.dm. metadata. batch.enabled	true/false	true	Specifies whether the batch job updates missing metadata fields (such as business_ friendly_name and value) on data ingestion.

Property	Valid Values	Default Value	Description
spring.kafka.dm.metadata.batch.chunk.size	Integer	200	Commits batch chunk size to update missing metadata fields (such as business_ friendly_name and value) on data ingestion.
spring.kafka.dm. metadata. bootstrap-servers	String (<host>:<port>)	localhost:9092	Specifies the Kafka broker addresses required for the DM consumer connection.
spring.kafka.dm. metadata. enable-auto-commit	True/False	TRUE	Enables or disables automatic offset commits.
spring.kafka.dm. metadata. group-id	String	dm-group-id	Assigns the consumer group ID for DM. Used for load balancing and fault tolerance.
spring.kafka.dm. metadata. listener-concurrency	Integer	2	Sets the number of concurrent DM consumer threads for parallel processing.
spring.kafka.dm. metadata. security-protocol	PLAINTEXT, SSL, SASL_PLAINTEXT, SASL_SSL	PLAINTEXT	Defines the security protocol for the DM consumer.
spring.kafka.dm. metadata. ssl-key-password	Key password (string)	Not set	Password for the SSL private key inside the keystore.

Property	Valid Values	Default Value	Description
spring.kafka.dm.metadata.ssl-key-store-location	Path to keystore file (e.g., /etc/ssl/kafka.keystore.jks)	Not set	Specifies the file location of the SSL keystore. Required for SSL connections.
spring.kafka.dm.metadata.ssl-key-store-password	Keystore password (string)	Not set	Password for accessing the SSL keystore.
spring.kafka.dm.metadata.ssl-trust-store-location	Path to truststore file (e.g., /etc/ssl/kafka.truststore.jks)	Not set	Specifies the file location of the SSL truststore. Required for SSL connections.
spring.kafka.dm.metadata.ssl-trust-store-password	Truststore password (string)	Not set	Password for accessing the SSL truststore.
spring.kafka.dm.metadata.topic	String	dm-topic	Defines the topic used for DM message consumption.
spring.kafka.listener.concurrency	Integer	50	Number of concurrent consumer threads for parallel processing.
spring.kafka.producer.acks	String	all	Sets the number of acknowledgments required for a message. Ensures reliability of delivery.

Property	Valid Values	Default Value	Description
spring.kafka. producer. batch-size	Integer	16384	Sets the maximum batch size in bytes. Used to optimize throughput.
spring.kafka. producer. client-id	String	data- enrichment -client	Assigns a unique ID to the producer client. Useful for tracking and logging.
spring.kafka. producer. key-serializer	Fully qualified class names of Kafka serializer implementations	org.apache. kafka. common. serialization. StringSerializer	Defines the serializer used for message keys. Required for producer configuration.
spring.kafka. producer. linger-ms	Integer	1	Specifies the time to wait before sending a batch. Used to improve batching efficiency.
spring.kafka. producer. properties. confluence. cred.source	USER_INFO, SASL_INHERIT	Not set	Specifies the credential source for Confluent Schema Registry when using the Confluent platform.
spring.kafka. producer. properties. connections. max.idle.ms	Integer	30000	Sets the maximum idle time (in ms) for producer connections before they are closed.

Property	Valid Values	Default Value	Description
spring.kafka.producer.properties.enable-idempotence	True/False	TRUE	Enables the idempotent producer to guarantee exactly-once message delivery.
spring.kafka.producer.properties.max-in-flight-requests-per-connection	Integer	1	Limits the number of concurrent requests per connection. Ensures ordering and supports idempotence.
spring.kafka.producer.properties.max.block.ms	Integer	10000	Defines the maximum time (in ms) a send() call blocks before throwing a timeout error.
spring.kafka.producer.properties.max.request.size	Integer	10485760	Specifies the maximum request size in bytes. Useful for handling large messages.
spring.kafka.producer.properties.sasl.jaas.config	JAAS config string for SASL authentication	Not set	Provides JAAS configuration for SASL authentication. Required when the security protocol is SASL.

Property	Valid Values	Default Value	Description
spring.kafka. producer. properties.sasl. mechanism	PLAIN, SCRAM-SHA-256, SCRAM-SHA-512, GSSAPI, etc.	Not set	Defines the SASL mechanism used for authentication. Applicable when SASL security is enabled.
spring.kafka. producer. properties.schema. registry.url	URL to your schema registry	Not set	Specifies the schema registry URL. Required when using Avro or Protobuf serialization.
spring.kafka. producer. properties. schema. registry.user.info	username:password for schema registry	Not set	Provides user credentials for schema registry authentication. Required when connecting to a secured registry.
spring.kafka. producer. properties. ssl.endpoint. identification. algorithm	https, empty string (disable)	Not set	Configures SSL endpoint identification. Use https for hostname verification, or an empty string to disable.
spring.kafka. producer.retries	Integer	3	Defines the number of retry attempts for failed sends. Provides fault tolerance.

Property	Valid Values	Default Value	Description
spring.kafka. producer. security.protocol	String	PLAINTEXT	Specifies the security protocol used for connections. Use PLAINTEXT for non-secure connections.
spring.kafka. producer.topic	String	transformed-data	Defines the topic to which the producer sends messages. Required for publishing.
spring.kafka. producer. value-serializer	Fully qualified class names of Kafka serializer implementations	org.apache. kafka. common. serialization. StringSerializer	Defines the serializer used for message values. Required for producer configuration.

7.13 Property Value Encryption

7.13.1 Overview

Decision applications support secure storage of sensitive configuration values—such as database passwords, API keys, and credentials—by allowing **encrypted properties** in configuration files (**application.yml** or **application.properties**). Encrypted property values enhance security by ensuring that sensitive information remains protected, even if configuration files are exposed.

7.13.2 How Property Encryption Works

- Encrypted properties must begin with the prefix ENC: to indicate they are securely encrypted.
- The encrypted value must be generated using the **Decision Encryption Tool**, which uses the RSA encryption algorithm.
- During runtime, the Decision application automatically decrypts these properties using the appropriate decryption key.

```
# Encrypted database password in application.properties
spring.datasource.password=ENC:lmclyieQY87EJD.....
```

7.13.3 Encryption Tool Usage

The Decision Encryption Tool (**decision-encrypt-2.0.jar**) is a command-line utility used to generate encrypted values using RSA encryption.

Supported Encryption Algorithm:

- RSA

7.13.4 Generating Encrypted Property Values

Step 1: Ensure Java is Installed

- Make sure JDK 17 or higher is installed and configured on your system.
- Set the JAVA_HOME environment variable if it's not already set.
- **Example**:

```
export JAVA_HOME="/path/to/jdk-17"
```

Step 2: Obtain the Encryption Tool

- Ensure the file **decision-encrypt-2.0.jar** is available on your system.

Step 3: Run the Encryption Tool

Use the command-line interface to generate an encrypted value.

- **Windows Command Example:**

```
java -jar decision-encrypt-2.0.jar RSA value-to-encrypt
```

- **Linux Command Example: bash Copy Edit**

```
java -jar decision-encrypt-2.0.jar 'RSA' 'value-to-encrypt'
```

  Replace value-to-encrypt with the actual sensitive value (e.g., a database password).

Step 4: Review the Tool Output

The encryption tool will output the following details:

- Entered Text: The value you entered for encryption
- Encryption Public Key: (for reference only)
- Encrypted Text: The encrypted value to use in your configuration file

Copy the Encrypted Text value.

Step 5: Add Encrypted Value to Configuration

Insert the encrypted value into your configuration file using the **ENC:** prefix.

- **Example (application.yml):**

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/decision_deanalytics
    username: dan_user
    password: ENC:lmclyieQY87EJD.....  # Encrypted password
```

- **Example (application.properties):**

```
spring.datasource.url=jdbc:postgresql://localhost:5432/decision_
deanalytics
spring.datasource.username=dan_user
spring.datasource.password=ENC:lmclyieQY87EJD.....
```

Step 6: Verify Decryption During Application Startup

- Start the Decision application.
- Check the application logs to confirm no decryption errors are reported.
- The application should automatically decrypt the values at runtime.

8. Snowflake Deployment & Installation

8.1 Overview

Decision supports multiple deployment styles, each distributed as a specific package. While the packages are functionally equivalent, they differ in:

- Layout
- Installation and configuration methods
- Platform dependencies
- Start up mechanisms and commands

8.2 Containerized Image Deployment

Containerized deployment packages the application components as Docker images, enabling flexibility, scalability, and simplified management across containerized platforms.

8.2.1 Overview

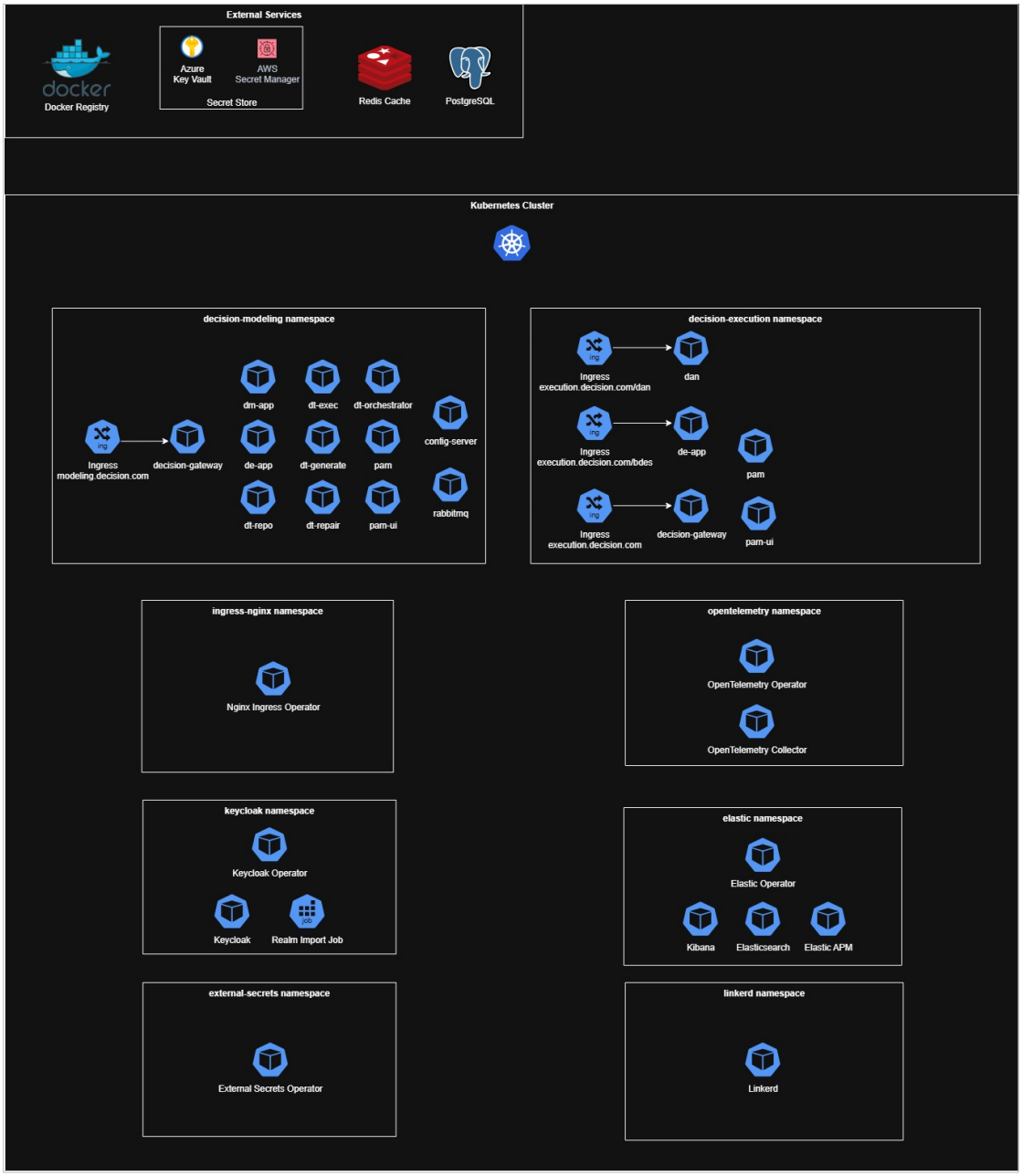
Containerized deployment is an advanced variant of standalone deployment. In this model, Sapiens provides application components as JAR files embedded within Docker images, ready to be deployed in environments such as:

- Kubernetes
- OpenShift
- Azure Kubernetes Service (AKS)
- Amazon Elastic Kubernetes Service (EKS)

For complete deployment steps, refer to the Helm chart's README.md file, as outlined in the Accessing the README.md File for Deployment Instructions section.

Advantages of Containerized Deployment:

- **Pre-Packaged Application JARs**: Application JARs are bundled within Docker images, simplifying deployment.
- **Isolated Failure Points**: A failure in one application component does not affect others, except when dependencies exist.
- **Flexibility and Scalability**: Components can be independently scaled and managed.
- **Fast Startup**: Each application component can be rapidly started in isolation.
- **Resource Optimization**: Independent memory and resource allocation is provided for each component, tailored to its requirements.
- **Enhanced Monitoring**: Each application runs as an independent Java process, enabling granular monitoring.
- **Fine-Grained Scaling**: Individual components can be scaled as needed.



8.2.2 Accessing the README.md File for Deployment Instructions

Follow the steps below to locate the Helm chart's README.md file:

- Locate the downloaded package file.
- Extract the package using any archive tool (e.g., WinRAR, 7-Zip).
- Navigate to the Snowflake directory in the extracted folder.
- Open the README.md file available at the root of the Snowflake folder.

```
total 64
-rw-r--r--   1    9230  1 Jul 19:47 values.yaml
-rw-r--r--   1   11484  1 Jul 19:47 README.md
-rw-r--r--   1     241  1 Jul 19:47 Chart.yaml
-rw-r--r--   1     239  1 Jul 19:47 Chart.lock
drwxr-xr-x  13     416 17 Jul 18:19 templates
drwxr-xr-x   3      96 17 Jul 18:19 charts
INLT-G7WMJLYK42-$
```

8.2.3 Package Content

The containerized deployment package includes the following essential components:

- Docker images and associated files for each application component.
- Helm charts for managing deployment in Kubernetes environments.

8.2.4 Recommended Use Cases

Containerized deployment is recommended in the following scenarios:

- Deployment to a containerized environment (e.g., Kubernetes, OpenShift, AKS, EKS) is required.
- A preference exists for pre-packaged Docker images for simplified deployment.
- Requirement for scalability, resource allocation, monitoring, and isolation at the per-application level.
- Environments requiring fast recovery and startup of each application component.
- High-availability production environments with a large number of users and/or high request rates.

8.2.5 Prerequisites

Ensure the following platform requirements are fulfilled before initiating containerized deployment:

- **Compatible Containerized Environment**: Kubernetes, OpenShift, AKS, or EKS.
- **Snowflake Requirements**:
  - Docker Image
  - Database schema granted full read/write permissions on all schema objects (e.g., tables, sequences, etc.). Refer to the Database Implementation section for database-specific instructions.
- **DE Requirements**:
  - Docker Image
  - Database schema granted full read/write permissions on all schema objects (e.g., tables, sequences, etc.). Refer to the Database Implementation section for database-specific instructions.
- **Decision Gateway**:

- Docker Image
- Redis Database.

- **Configuration Service (Optional)**:
  - **Rabbit MQ**:
    - **Docker Image for RabbitMQ**: This is a pre-built, standardized package that allows you to run RabbitMQ easily using Docker.
    - **Tag: 3-management**: This indicates a specific version of the RabbitMQ Docker image, with the "management" plugin enabled. The management plugin provides a user-friendly web-based management UI for RabbitMQ.
- **OAuth IDP (e.g., Keycloak)**: Required for user authentication using OAuth 2.0.
- **Ingress NGINX Controller**: Used for managing external access.
- **APM Tool (Optional)**: Application Performance Monitoring (e.g., Elastic ECK).
- **External Secrets Operator (Optional)**
- **Open Telemetry Configuration (Optional)**:

  OpenTelemetry is an open-source observability framework for generating, collecting, processing, and exporting telemetry data (such as traces, metrics, and logs) from applications.

  Components of OpenTelemetry Configuration:
  - OpenTelemetry Operator
  - OpenTelemetry Collector
  - Auto-Instrumentation Configuration

8.2.6 Installation

For detailed installation instructions, refer to the Helm chart's README.md file, as explained in the Accessing the README.md File for Deployment Instructions section.

8.3 All-in-One Deployment

The all-in-one deployment model is a traditional approach where all Decision components are deployed on a single host machine, operating under one application server (such as Tomcat). This approach simplifies management but has limitations in terms of scalability and resource optimization.

8.3.1 Overview

The following are brief descriptions of prerequisite directories included in the current Snowflake release and mentioned throughout this document.

Folder	Description
Conf	Configuration files for Snowflake configuration

Folder	Description
War	App WAR file to be installed on the Tomcat Application Server
Release Notes	The Release Notes document

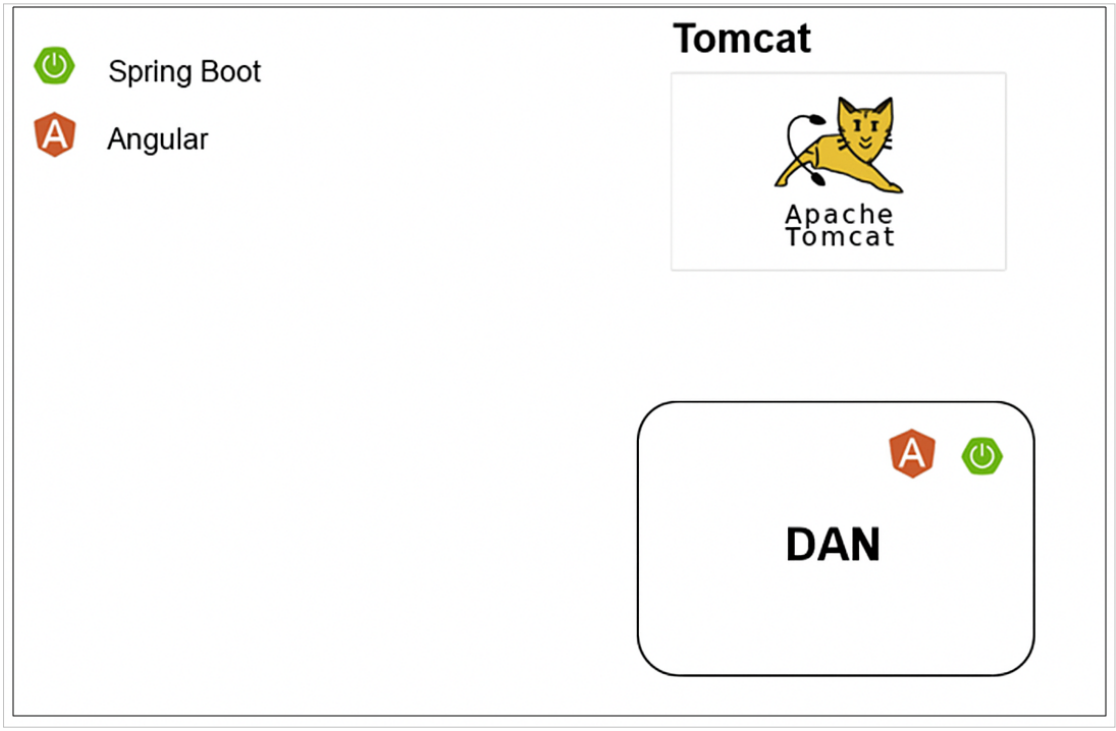
Note: Once the installation is complete and Tomcat is started, a Snowflake instance will be running.

Advantages:

- Centralized configuration and maintenance for all Decision components.
- No network latency between components, as they run on the same host.

Disadvantages:

- Single point of failure – an issue with one component can impact the entire application.
- Lack of fine-grained resource allocation for individual components.
- Larger application archive and memory footprint.
- Slower scaling, as all components must be scaled together.



All-in-One deployment Architecture

8.3.2 Package Content

This section outlines the components included in the all-in-one deployment package:

- **Web Application Archive (WAR)**: Application components are packaged as WAR files, designed to be deployed on a single application server (e.g., Apache Tomcat).

8.3.3 Recommended Use Cases

The all-in-one deployment method is recommended in the following scenarios:

- The customer prefers to host the Decision application on an existing web application server.
- When centralized configuration through a single application server, unified deployment location, and single-port access is prioritized over:
    - Per-application instance scalability
    - Dedicated resource allocation
    - Isolated monitoring
    - Component-level isolation

8.3.4 Prerequisites

This section outlines the essential system and platform requirements for deploying Decision using the war deployment model. Ensure all the following prerequisites are met before starting the installation.

Platform Requirements:

- **Application Server:**
    - Tomcat Application Server (version 10 or above)
- **Java Development Kit (JDK):**
    - Installed JDK distribution of Java 17
    - The JDK installation path should be set in the **JAVA_HOME** environment variable, either by the operating system or directly within the Tomcat application server.
- **Disk Space:**
    - Minimum of **2GB** of available disk space (including the Tomcat server).
- **Memory (RAM):**
    - For basic usage (application startup): At least 3GB of RAM.
    - For optimal performance in most production environments: At least 8GB of RAM

Database Requirements:

- **Database Server:**
    - PostgreSQL (refer to each application installation guide for minimum supported versions and database-specific instructions).
- **Dedicated Database Schema:**
    - Each Decision application must have a dedicated schema with full read/write permissions on all schema objects (tables, sequences, etc.).
    - Multiple schemas can exist under the same database server instance, but each must be isolated per application.

Application Package (WAR Files):

The following web application archive (WAR) files are required for a complete WAR deployment:

- Mandatory Components:
    - **snowflake.war** - Snowflake Application

8.3.5 Installation

This section provides a comprehensive, step-by-step guide to install the Decision components in a WAR deployment using the Apache Tomcat application server. Ensure that all prerequisites (Java, Tomcat, WAR files, and database setup) are fulfilled before proceeding.

Step 1: Set Up Global Classpath and Configuration Folders

1. **Edit Tomcat Global Configuration**
    - Navigate to the Tomcat configuration directory:
      <Tomcat folder>/conf/catalina.properties
    - Locate the shared.loader property and append the Decision configuration directory path:

      ```
      shared.loader=${catalina.home}/decision-config
      ```

    - This allows Tomcat to load shared configurations across all deployed WAR applications.
2. **Create the Configuration Directory**
    - If it does not already exist, create a directory named **decision-config** in the Tomcat root directory.
3. **Create Configuration Sub-Directories per Application**
    - Inside the decision-config directory, create the following sub-directory for Snowflake application:
        - snowflake-config
4. **Add Configuration Files**
    - Within the application's subdirectory, add your application configuration file:
        - Either **application.properties** or **application.yml**, depending on your format preference.

Step 2: Configure Application Contexts in Tomcat

1. **Edit the server.xml File**
    - Open the following file: <Tomcat folder>/conf/server.xml
    - Inside the <Host> tag, add a <Context> entry for each Decision application. For example:

```
<Host
name
="localhost"

appBase
="webapps"
 unpackWARs="true" autoDeploy="true" startStopThreads="10">
    <Context docBase="dan" path="/dan" />
    <Context docBase="dm" path="/dm" />
    <Context docBase="dt" path="/dt" />
    <Context docBase="bdes" path="/bdes" />
</Host>
```

- The docBase should match the WAR file name (without the .war extension).
- The path defines the application's context URL.

2. **Adjust Start-Stop Threads:**
   - Set the startStopThreads attribute based on the number of <Context> entries.
   - The recommended value is equal to the total number of applications being deployed.

Step 3: Configure HTTP and Shutdown Ports

1. **Set HTTP Port:**
   - Still in server.xml, modify the HTTP <Connector> tag as needed. For example, to use port 9080, update the configuration accordingly. The default port is 8080.

```
<Connector port="9080" protocol="HTTP/1.1"
           connectionTimeout="20000"
           redirectPort="8443"
           maxParameterCount="1000" />
```

2. **Set Shutdown Port**
   - Change the shutdown port to avoid conflicts with other local servers. The default shutdown port is 8005.

```
<Server port="9005" shutdown="SHUTDOWN">
```

   Avoid using the default 8005 if multiple Tomcat instances are running on the same host.

Step 4: Create and Configure setenv.sh Script for Tomcat

1. **Create the setenv.sh Script**
   - If not already present, create a setenv.sh file in the <Tomcat folder>/bin directory.
2. **Add Environment Variables and JVM Options**

   Edit **setenv.sh** and add the following content:

```
#!/bin/sh

export JAVA_HOME="/home/.jdks/corretto-17.0.14"

export JAVA_OPTS="-Xmx8g \
-Ddecision.home=${catalina.home} \
-Ddecision.dan.home=${catalina.home} \
-Dsun.io.useCanonCaches=true \
-XX:+UseParallelGC \
-Dfile.encoding=UTF-8 \
-Dcom.sun.net.ssl.enableECC=true \
--add-opens=java.base/sun.reflect.annotation=ALL-UNNAMED \
--add-modules=java.se \
--add-exports=java.base/jdk.internal.ref=ALL-UNNAMED \
--add-opens=java.base/sun.nio.ch=ALL-UNNAMED \
--add-opens=java.base/java.lang=ALL-UNNAMED \
--add-opens=java.management/sun.management=ALL-UNNAMED \
--add-opens=jdk.management/com.sun.management.internal=ALL-UNNAMED \
--add-opens=java.base/java.util=ALL-UNNAMED"
```

3. **Edit Configuration Based on Your Environment**
    - Set **JAVA_HOME** to the absolute path of your installed Java 17 JDK.
    - Modify the **-Xmx8g** flag to allocate appropriate maximum heap memory depending on system resources.

Step 5: Prepare Database Schemas

Before deploying the applications, ensure the database environment is ready.

- Verify that database schemas for each Decision application (Snowflake, DE, DM and DT) are created.
- Each schema should have sufficient read/write privileges, as defined in the Prerequisites section of this guide.

Step 6: Deploy Application WAR Files

1. **Stop Running Tomcat Server**
    - If any Decision applications are running on the target Tomcat server, stop the server.
2. **Copy WAR Files**
    - Copy the Decision application WAR files to the **<Tomcat folder>/webapps** directory.
3. **Rename WAR Files**
    - Remove the version numbers from the WAR filenames:

      e.g., snowflake-1.0.war → snowflake.war
4. **Clean Up Existing Files**
    - Delete any existing WAR files or folders of a Decision application you are about to redeploy.

Step 7: Configure Application Properties

- Refer to the Configuration section of this document.
- Populate the configuration file with all required properties under the section Customized Configuration Properties.

8.3.6 Snowflake Server – Starting and Stopping

Note: Before starting the server, you can validate that all configurations are correct by running the version.sh script.

Example: {TOMCAT_HOME}/bin/version.sh

To Start the Snowflake Server:

- Run:

```
{TOMCAT_HOME}/bin/startup.sh
```

To Stop the Snowflake Server:

- Run

```
{TOMCAT_HOME}/bin/shutdown.sh
```

Note: To verify that the server is running without any errors, use the following curl command: (Assuming the default context path /snowflake)

Example:

```
curl http://<host>:<port>/dan/actuator/health
```

If the connection fails, possible causes include:

- Incorrect port, IP address, or DNS
- Firewall or security group blocking access

Check the following logs for more details:

- Snowflake logs: snowflake/log
- Tomcat logs: {TOMCAT_HOME}/logs

8.3.7 Upgrade Procedure

To upgrade Snowflake to the current version, perform the following steps:

1. Stop the application server.
2. Back up your data, database, and/or file system repository.
3. Back up the **snowflake** directory and **snowflake.war** file located in **{tomcat_ home}/webapps**.
4. Delete the **snowflake** directory and **snowflake** file from **{tomcat_home}/webapps**.

5. Copy the new **snowflake.war** file from the release package location to **{tomcat_ home}/webapps**.
6. Ensure the configuration file **{snowflake_home}/conf/snowflake-config/application.properties** is updated as needed.
7. Start the upgraded Snowflake server and review the Snowflake logs to ensure there are no errors. Schema changes for the current version will be applied automatically, and an audit of these changes can be found in the Snowflake logs or in the Snowflake database table **flyway_schema_history**.
8. Once the Snowflake application is running, perform a connection test using a web browser or curl.

```
curl http://<host>:<port>/dan/actuator/health
```

8.4 Standalone Deployment

8.4.1 Overview

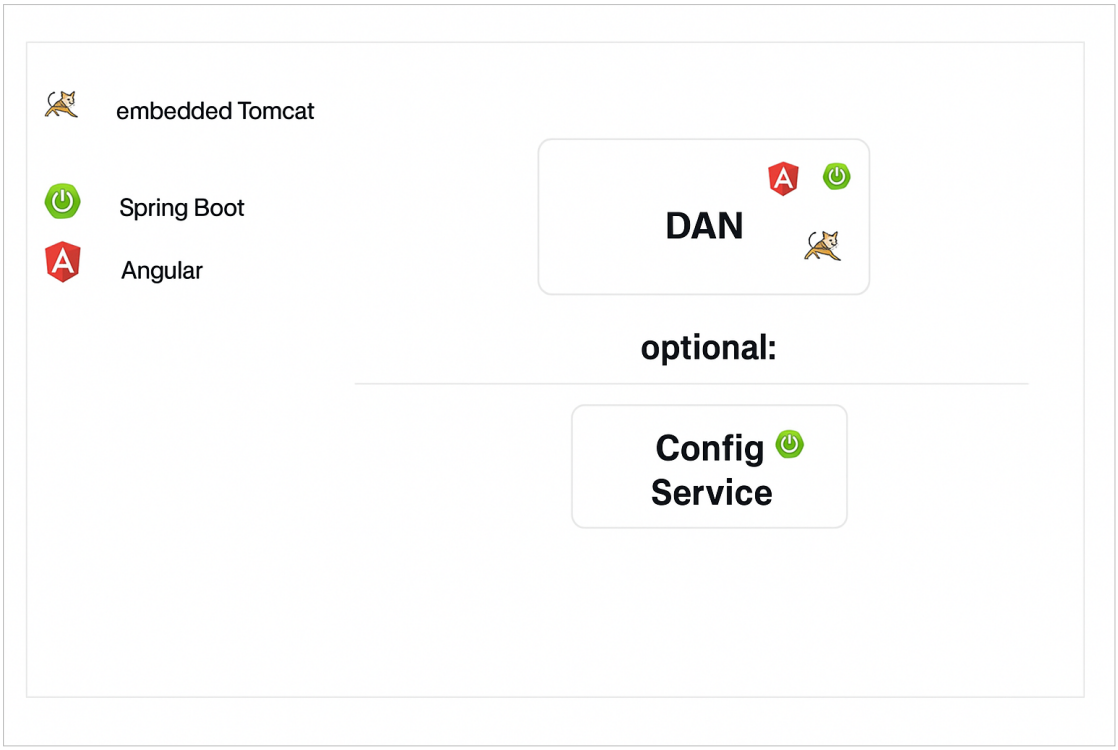
In standalone deployment, each Decision application is packaged as a self-executable JAR file generated using the Spring Boot framework. These JAR files contain an embedded Tomcat server, making them completely independent and self-sufficient.

Key Characteristics:

- **Independent Execution:**
  - Each application runs on its own port and can be hosted on the same or different machines.
- **Application-Specific Configuration:**
  - Server-related properties (e.g., port, SSL) are managed directly in the application's configuration file (application.properties or application.yml) or via Java command-line arguments.
- **Improved Flexibility and Scalability:**
  - This model allows fine-grained control over resource allocation, monitoring, and scaling for each application.

Advantages:

- Distributed points of failure — a failure in one component does not impact others.
- Maximum flexibility, scalability, and isolation between application components.
- Fast startup due to isolated application instances.
- Independent memory and resource allocation for each component.
- Per-application monitoring using separate Java processes.
- Fine-grained scaling for each application.



Snowflake Standalone Deployment Architecture

8.4.2 Package Content

The standalone deployment package includes the following components:

- **Application Java Binary Files:**

  Compiled Java class files for each Decision application.
- **Spring Boot Framework Files:**

  Essential Spring Boot classes required for application startup and management.
- **Java Third-Party Libraries:**

  External JAR files used by the application for various dependencies.
- **Maven Build Metadata:**

  Configuration details, including project structure, dependencies, and build information.

8.4.3 Recommended Use Cases

Standalone deployment is ideal in the following scenarios:

- When you prefer to build your own container images using the application JAR files.
- When fine-grained scalability, resource allocation, monitoring, and isolation per application are required.
- When fast recovery and startup times for each application are critical.

Common Use Cases:

- Production environments with a large number of users.
- High-traffic scenarios involving frequent server requests.

8.4.4 Prerequisites

The standalone deployment of Decision requires specific platform dependencies for each component to ensure optimal performance and smooth operation. These prerequisites include Java installation, available disk space, memory allocation, database setup, and connectivity between components.

Java Installation (Global Requirement)

- **JDK Version:**

  Installed JDK distribution of Java 17.

- **JAVA_HOME Configuration:**

  Ensure that the Java installation path is correctly set in the JAVA_HOME environment variable, either at the operating system level or through the application's Java command line.

- **Memory Allocation (-Xmx):**

  The memory allocation size for each application should be determined based on the expected user load and the number of concurrent requests. This can be configured via Java command-line arguments.

  Example:

  ```
  java -Xmx1g -jar <app_jar_file_path>
  ```

- **Disk Space (Minimum Requirements)**

  Each application has specific disk space requirements, which are listed below (list to be added as per actual application specs).

Component	Disk Space	Memory (Xmx)	Additional Dependencies
Snowflake	185 MB	Minimum: 8 GB	Dedicated database schema (PostgreSQL).

8.4.5 Installation

To install any Decision application in standalone mode, follow the steps below:

1. **Database Schema Setup**
   - If the application requires a database schema, ensure it already exists.
   - If the schema does not exist, create it before proceeding.
2. **Application JAR Placement**
   - Copy the self-executable JAR file of the application to your desired application home folder.
3. **Configuration File Setup**
   - Create or copy the appropriate configuration file (application.properties or application.yml).
   - Ensure this file is placed in the same folder as the application JAR file.

4. **Configuration Settings**
    - Refer to the Configuration section of this document.
    - Populate the configuration file with all required properties under the section Customized Configuration Properties.

8.5 Post Installation Validation

8.5.1 Health Check Validation

The following endpoint is available to perform heartbeat and health checks for the Snowflake application:

- https://<host>:<port>/snowflake/actuator/health

  Example: http://localhost:9081/snowflake/actuator/health

8.5.2 Snowflake Startup Process and Logs

Depending on the deployment setup, the Snowflake application will start as part of the Tomcat initialization process.

Once the Snowflake application starts, log files will be created at the location specified by the **logging.file.path** variable. Additional log files can also be found in the **{TOMCAT_HOME}/logs** directory.

The primary Snowflake log file is named **snowflake.log** and contains most runtime information. Some additional logs may be stored in other files depending on configuration.

Sample Snowflake log entries at startup (actual messages may vary):

- **INFO [ServletWebServerApplicationContext] Root WebApplicationContext**: initialization started
- **INFO [c.z.h.HikariDataSource] [] HikariPool-1 - Starting...** (Indicates database connection initialization)
- **INFO [ServletWebServerApplicationContext] Root WebApplicationContext**: initialization completed in xxx ms (Marks completion of the startup process)

9. Snowflake Database Implementation (Optional)

The following implementation guide is for unencrypted access. For encrypted access, refer to the Property Value Encryption section.

9.1 PostgreSQL

Follow these steps to configure PostgreSQL for the Snowflake application:

1. **Edit application.properties**

   Update or create the application's application.properties file

2. **Define or Use an Existing Database User**

   You can either create a new user or use an existing one (e.g., SnowflakePostgresUser).

3. **Update Configuration Placeholders**

   Replace the following placeholders in the application.properties file:

   - spring.datasource.url – Specify the JDBC URL of the database
   - spring.datasource.username – Specify the database username
   - spring.datasource.password – Specify the database user's password.

   Example Configuration:

   ```
   spring.datasource.url=jdbc:postgresql://localhost:5432/
   {databaseName}?currentSchema=dan
   spring.datasource.username=DanPostgresUser
   spring.datasource.password=DanPostgresPass
   ```

4. **Create the Database**

   Create a PostgreSQL database if it does not already exist.

5. **Create the Snowflake Schema**

   Within the PostgreSQL database, it is mandatory to create the Snowflake application schema under default postgres database for the Snowflake application.

10. Snowflake Configuration

Configuration Properties inSnowflake Server are settings that determine how the system behaves. This section describes the customizable properties that system administrators can use to control the behavior of Snowflake Server components. Only authorized personnel should modify the contents of the property files.

10.1 Multi-Thread Pool Configuration

The multi-thread configurations define thread pools for concurrent task execution in your Spring Boot application. Improves application throughput and responsiveness by enabling parallel processing of tasks, reducing bottlenecks and optimizing resource usage.

10.1.1 Message Thread Pool Configuration

These properties configure the thread pool for message processing, including core and maximum thread counts and queue capacity, to optimize concurrency and resource usage.

Name	Valid Values	Default Value	Description
decision.dan.snowflake.threadpool.message.core-size	Integer	25	Specifies the minimum number of threads always kept alive for message processing.
decision.dan.snowflake.threadpool.message.max-size	Integer	50	Defines the maximum number of threads allowed in the message thread pool.
decision.dan.snowflake.threadpool.message.queue-capacity	Integer	10	Sets the number of tasks that can wait in the queue before new threads are created.

10.1.2 Async Database Thread Pool Configuration

These properties configure the thread pool for asynchronous database operations, including core and maximum thread counts and queue capacity, to optimize concurrent DB task execution.

Name	Valid Values	Default Value	Description
decision.dan.snowflake.threadpool.persistence.core-size	Integer	20	Specifies the minimum number of threads always kept alive for database tasks.
decision.dan.snowflake.threadpool.persistence.max-size	Integer	20	Defines the maximum number of threads allowed in the database thread pool.
decision.dan.snowflake.threadpool.persistence.queue-capacity	Integer	200	Sets the number of database tasks that can wait in the queue before new threads are created.

10.1.3 Resolver Thread Pool Configuration

These properties configure the thread pool for resolving dimension entities, including core and maximum thread counts and queue capacity, to optimize concurrent DB task execution.

Name	Valid Values	Default Value	Description
decision.dan.snowflake.threadpool.resolver.core-size	Integer	2	Specifies the minimum number of threads always kept alive for database tasks.
decision.dan.snowflake.threadpool.resolver.max-size	Integer	4	Defines the maximum number of threads allowed in the database thread pool.
decision.dan.snowflake.threadpool.persistence.queue-capacity	Integer	200	Sets the number of database tasks that can wait in the queue before new threads are created.

10.2 Health Check Configuration

These properties control the health check endpoints to monitor the application's status. They define the endpoints, configure timeouts, and specify which health details and probes are exposed through Spring Boot Actuator. System administrators can disable certain checks (such as Database or SSL) to customize what is reported, based on operational requirements.

Name	Valid Values	Default Value	Description
management.endpoint.health.probes.enabled	True/False	true	Enables or disables health probes.
management.endpoint.health.show-details	String	always	Controls the level of health details displayed.

Name	Valid Values	Default Value	Description
management.endpoints. web.exposure.include	String	health	Specifies which Actuator endpoints are exposed.
management.health. db.enabled	True/False	false	Enables or disables the database health check.
management. health.defaults.enabled	True/False	false	Enables or disables default health checks.
management.health .ssl.enabled	True/False	false	Enables or disables the SSL health check.

The DAN Snowflake application also provides a distributed, event-driven health monitoring system built on Kafka. It enables microservices to publish sanitized health metrics to a central Kafka topic, allowing external systems to consume real-time health data without direct polling.

Name	Valid Values	Default Value	Description
decision.dan.snowflake.health.check.remove.items	Comma-separated field names	validationQuery, path	Specifies sensitive fields to be removed from health check JSON output before publishing to Kafka. validationQuery removes database validation SQL queries, path removes file system paths that could expose internal structure

Name	Valid Values	Default Value	Description
decision.dan.snowflake.health.check.topic	String (Kafka topic name)	heart-beat	Kafka topic name where sanitized health check data is published. heart-beat topic allows external monitoring systems to track application health status without direct HTTP access.
decision.dan.snowflake.health.scheduler.frequency.rate	Milliseconds (positive integer)	60000	Interval in milliseconds between scheduled health check publications to Kafka. 60000ms (1 minute) provides regular health monitoring without overwhelming the Kafka cluster with frequent messages.

Name	Valid Values	Default Value	Description
decision.dan.snowflake.health.scheduler.enabled	Boolean	false	Controls whether the scheduled health check publisher is active. Currently false for local development to avoid Kafka topic pollution. Should be true in production for continuous health monitoring.

10.3 DataSource Configuration

These properties define how the application connects to and manages the database, including pool-specific overrides, Hibernate settings, Flyway migrations, transaction persistence, and performance optimizations to ensure efficient and reliable data operations.

Name	Valid Values	Default Value	Description
Basic Datasource Configuration			
spring.datasource.url	JDBC URL format	jdbc:postgresql://localhost:5432/dan_snowflake?currentSchema=dan	PostgreSQL database connection URL with query parameters
spring.datasource.username	String	dans	Database username for authentication

Name	Valid Values	Default Value	Description
spring.datasource.password	String/Encrypted	sapiens	Database password (encrypted with Jasypt)
spring.datasource.driver-class-name	Class name	org.postgresql.Driver	JDBC driver class for PostgreSQL
HikariCP Connection Pool			
spring.datasource.hikari.pool-name	String	DanSnowflakeHikariCP	Name identifier for the connection pool
spring.datasource.hikari.maximum-pool-size	Integer (1-∞)	8	Maximum number of connections in pool
spring.datasource.hikari.minimum-idle	Integer (0-max-pool-size)	2	Minimum number of idle connections
spring.datasource.hikari.connection-timeout	Milliseconds	10000	Maximum time to wait for connection from pool
spring.datasource.hikari.validation-timeout	Milliseconds	2000	Maximum time for connection validation

Name	Valid Values	Default Value	Description
spring.datasource.hikari.idle-timeout	Milliseconds	300000	Maximum idle time before connection retirement
spring.datasource.hikari.max-lifetime	Milliseconds	1800000	Maximum lifetime of connection in pool
spring.datasource.hikari.leak-detection-threshold	Milliseconds (0=disabled)	0	Time threshold for connection leak detection
spring.datasource.hikari.auto-commit	Boolean	false	Auto-commit behavior for connections
spring.datasource.hikari.login-timeout	Seconds	10	Maximum time to wait for database login
spring.datasource.hikari.transaction-isolation	Isolation level	TRANSACTION_READ_COMMITTED	Transaction isolation level
spring.datasource.hikari.keepalive-time	Milliseconds	300000	Frequency of keepalive ping to maintain connections
JDBC Driver Properties			

Name	Valid Values	Default Value	Description
spring.datasource.hikari.data-source-properties.reWriteBatchedInserts	Boolean	true	Enable PostgreSQL batch insert optimization
spring.datasource.hikari.data-source-properties.binaryTransfer	Boolean	true	Enable binary format for data transfer
spring.datasource.hikari.data-source-properties.prepareThreshold	Integer (0-∞)	0	Threshold for prepared statement caching
spring.datasource.hikari.data-source-properties.stringtype	String	unspecified	Default string type mapping
spring.datasource.hikari.data-source-properties.defaultRowFetchSize	Integer	1000	Default number of rows to fetch per round trip
spring.datasource.hikari.data-source-properties.tcpKeepAlive	Boolean	true	Enable TCP keepalive for socket connections
spring.datasource.hikari.data-source-properties.socketTimeout	Milliseconds	30000	Socket timeout for database operations
spring.datasource.hikari.data-source-properties.connectTimeout	Milliseconds	10000	Connection timeout to database

Name	Valid Values	Default Value	Description
spring.datasource.hikari.data-source-properties.ApplicationName	String	DAN-Snowflake	Application name reported to PostgreSQL
Flyway Migration			
spring.flyway.enabled	Boolean	true	Enable/disable Flyway database migrations
spring.flyway.locations	Comma-separated paths	classpath:db/migration	Locations to scan for migration scripts
spring.flyway.baseline-on-migrate	Boolean	true	Baseline database on first migration
JPA/Hibernate Configuration			
spring.transaction.default-timeout	Seconds	30	Default timeout for transactions
spring.jpa.hibernate.ddl-auto	none, validate, update, create, create-drop	none	Hibernate DDL generation strategy
spring.jpa.show-sql	Boolean	false	Log SQL statements to console

Name	Valid Values	Default Value	Description
spring.jpa.open-in-view	Boolean	false	Enable Open Session In View pattern
spring.jpa.properties.hibernate.jdbc.fetch_size	Integer	2000	Number of rows fetched in single database trip
spring.jpa.properties.hibernate.jdbc.batch_size	Integer	200	Number of statements batched together
spring.jpa.properties.hibernate.order_inserts	Boolean	true	Order INSERT statements for better batching
spring.jpa.properties.hibernate.order_updates	Boolean	true	Order UPDATE statements for better batching
spring.jpa.properties.hibernate.cache.use_second_level_cache	Boolean	false	Enable Hibernate second-level cache
spring.jpa.properties.hibernate.cache.use_query_cache	Boolean	false	Enable Hibernate query result cache

Name	Valid Values	Default Value	Description
spring.jpa.properties.hibernate.connection.provider_disables_autocommit	Boolean	true	Hint that connection pool disables auto-commit
spring.jpa.properties.hibernate.connection.autocommit	Boolean	false	Auto-commit setting for Hibernate connections
PostgreSQL COPY Operations			
decision.dan.copy.synchronous-commit	ON, OFF, LOCAL, REMOTE_WRITE, REMOTE_APPLY	OFF	PostgreSQL synchronous commit level for COPY
decision.dan.copy.buffer-mb	Integer (MB)	64	Buffer size for COPY operations
COPY Batching Configuration			
decision.dan.snowflake.copy.maxContainersPerBatch	Integer	500	Maximum containers processed per batch
decision.dan.snowflake.copy.maxFactsPerBatch	Integer	20000	Maximum facts processed per batch
decision.dan.snowflake.copy.maxInFlight	Integer	2	Maximum concurrent COPY operations

10.4 Jasypt Configuration

Refer to the Jasypt Configuration section for more details.

10.5 Environment and Server Configuration

Refer to the Environment and Server Configuration section for more details.

10.6 Kafka Configuration Properties

Kafka properties in Snowflake are required to enable efficient, scalable, and reliable communication between microservices for processing, transferring, and consuming large volumes of real-time data. They configure how Snowflake connects to Kafka brokers, manages message production and consumption, ensures data integrity, and supports concurrency and fault tolerance in distributed environments.

Property	Valid Values	Default Value	Descrip-tion
Core Kafka Configuration			
spring.kafka.bootstrap-servers	Com ma-separ ated host:p ort	localhost:9092	List of Kafka broker addresse s
Main Consumer Configuration			
spring.kafka.consumer.bootstrap-servers	Com ma-separ ated host:p ort	${spring.kafka.bootstrap-servers}	Bootstra p servers for main consume r
spring.kafka.consumer.topic	String	de-raw-data	Topic name for main consume r
spring.kafka.consumer.group-id	String	de-client-grp-2	Consum er group identifier

Property	Valid Values	Default Value	Descrip-tion
spring.kafka.consumer.client-id	String	de-client-2	Client identifier for this consume r
spring.kafka.consumer.auto-offset-reset	earlie st, latest, none	earliest	What to do when no initial offset or offset out of range
spring.kafka.consumer.enable-auto-commit	Boole an	false	Enable automati c offset commits
spring.kafka.consumer.key-deserializer	Class name	org.apache.kafka.common.se rialization.StringDeserializer	Key deseriali zer class
spring.kafka.consumer.value-deserializer	Class name	org.apache.kafka.common.se rialization.StringDeserializer	Value deseriali zer class
spring.kafka.consumer.security.protoc ol	PLAIN TEXT, SSL, SASL_ PLAIN TEXT, SASL_ SSL	PLAINTEXT	Security protocol for consume r
Main Consumer Performance Settings			
spring.kafka.consumer.max-poll-records	Intege r	200	Maximu m records returned in single poll

Property	Valid Values	Default Value	Descrip-tion
spring.kafka.consumer.fetch-min-size	Bytes	65536	Minimum data fetched per request
spring.kafka.consumer.fetch-max-wait	Milliseconds	100	Maximum time to wait for fetch-min-size
spring.kafka.consumer.session-timeout-ms	Milliseconds	45000	Session timeout for consumer group management
spring.kafka.consumer.heartbeat-interval-ms	Milliseconds	15000	Heartbeat interval to coordinator
spring.kafka.consumer.max-poll-interval-ms	Milliseconds	300000	Maximum time between polls before leaving group
spring.kafka.consumer.receive-buffer-bytes	Bytes	262144	TCP receive buffer size
spring.kafka.consumer.send-buffer-bytes	Bytes	131072	TCP send buffer size
Main Consumer Advanced Properties			

Property	Valid Values	Default Value	Descrip-tion
spring.kafka.consumer.properties.max.partition.fetch.bytes	Bytes	1048576	Maximum data per partition fetch
spring.kafka.consumer.properties.request.timeout.ms	Milliseconds	15000	Request timeout for consumer
spring.kafka.consumer.properties.retry.backoff.ms	Milliseconds	25	Backoff time between retry attempts
spring.kafka.consumer.properties.reconnect.backoff.ms	Milliseconds	25	Backoff time for reconnection attempts
spring.kafka.consumer.properties.reconnect.backoff.max.ms	Milliseconds	500	Maximum backoff time for reconnections
spring.kafka.consumer.properties.connections.max.idle.ms	Milliseconds	300000	Maximum idle time for connections
Listener Configuration			

Property	Valid Values	Default Value	Descrip-tion
spring.kafka.listener.ack-mode	RECORD, BATCH, TIME, COUNT, COUNT_TIME, MANUAL, MANUAL_IMMEDIATE	MANUAL_IMMEDIATE	Acknowledgment mode for message processing
spring.kafka.listener.concurrency	Integer	4	Number of consumer threads
spring.kafka.listener.poll-timeout	Milliseconds	1000	Timeout for consumer poll operations
spring.kafka.listener.idle-event-interval	Milliseconds	10000	Interval for idle events
spring.kafka.listener.missing-topics-fatal	Boolean	false	Whether missing topics should cause fatal errors
spring.kafka.listener.immediate-stop	Boolean	false	Whether to stop immediately on shutdown

Property	Valid Values	Default Value	Descrip-tion
DM (Asset Metadata) Consumer Configuration			
spring.kafka.dm.metadata.bootstrap-servers	Comma-separated host:port	${spring.kafka.bootstrap-servers}	Bootstrap servers for DM consumer
spring.kafka.dm.metadata.topic	String	dm-topic	Topic name for DM metadata
spring.kafka.dm.metadata.group-id	String	dm-group-id-2	Consumer group for DM metadata
spring.kafka.dm.metadata.auto-offset-reset	earliest, latest, none	earliest	Offset reset strategy for DM consumer
spring.kafka.dm.metadata.enable-auto-commit	Boolean	true	Enable auto-commit for DM consumer
spring.kafka.dm.metadata.security-protocol	PLAINTEXT, SSL, SASL_PLAINTEXT, SASL_SSL	PLAINTEXT	Security protocol for DM consumer

Property	Valid Values	Default Value	Descrip-tion
spring.kafka.dm.metadata.listener-concurrency	Integer	4	Number of listener threads for DM consumer
DM Consumer Batch Processing			
spring.kafka.dm.metadata.batch.enabled	Boolean	true	Enable batch processing for DM consumer
spring.kafka.dm.metadata.batch.chunk.size	Integer	50	Batch chunk size for DM processing
Main Consumer Security - MSK/SASL (Optional - Commented)			
spring.kafka.consumer.properties.sasl.jaas.config	JAAS config string	None	SASL JAAS configuration for AWS MSK IAM
spring.kafka.consumer.properties.sasl.mechanism	String	None	SASL mechanism for authentication

Property	Valid Values	Default Value	Descrip-tion
spring.kafka.consumer.security.protocol	PLAINTEXT, SSL, SASL_PLAINTEXT, SASL_SSL	None	Security protocol when using SASL
spring.kafka.consumer.properties.sasl.client.callback.handler.class	Class name	None	SASL callback handler for AWS MSK
spring.kafka.consumer.properties.ssl.endpoint.identification.algorithm	String	None	SSL endpoint identification algorithm
Main Consumer Security - SSL (Optional - Commented)			
spring.kafka.consumer.properties.ssl.truststore.location	File path	None	SSL truststore location
spring.kafka.consumer.properties.ssl.truststore.password	String	None	SSL truststore password
spring.kafka.consumer.properties.ssl.truststore.type	String	JKS (Kafka default)	SSL truststore type
spring.kafka.consumer.properties.ssl.protocol	String	TLSv1.3 (Kafka default)	SSL protocol version
spring.kafka.consumer.properties.ssl.enabled.protocols	Comma-separated	TLSv1.3,TLSv1.2 (Kafka default)	Enabled SSL protocol versions

Property	Valid Values	Default Value	Descrip-tion
Main Consumer Security - SCRAM (Optional - Commented)			
spring.kafka.consumer.properties.sasl.mechanism	None	None	SASL mechanism for SCRAM authentication
spring.kafka.consumer.properties.sasl.jaas.config	JAAS config string	None	SASL JAAS configuration for SCRAM
spring.kafka.consumer.properties.client.dns.lookup	None	use_all_dns_ips (Kafka default)	DNS lookup strategy
DM Consumer Security - MSK/SASL (Optional - Commented)			
spring.kafka.dm.metadata.properties.sasl.jaas.config	JAAS config string	None	SASL JAAS configuration for DM consumer
spring.kafka.dm.metadata.properties.sasl.mechanism	String	None	SASL mechanism for DM consumer
spring.kafka.dm.metadata.security-protocol	PLAINTEXT, SSL, SASL_PLAINTEXT, SASL_SSL	None	Security protocol for DM consumer

Property	Valid Values	Default Value	Descrip-tion
spring.kafka.dm.metadata.properties.sasl.client.callback.handler.class	Class name	None	SASL callback handler for DM consumer
spring.kafka.dm.metadata.properties.ssl.endpoint.identification.algorithm	String	None	SSL endpoint identification for DM consumer
DM Consumer Security - SSL (Optional - Commented)			
spring.kafka.dm.metadata.ssl-trust-store-location	File path	None	SSL truststore location for DM consumer
spring.kafka.dm.metadata.ssl-trust-store-password	String	None	SSL truststore password for DM consumer
spring.kafka.dm.metadata.ssl-key-store-location	File path	None	SSL keystore location for DM consumer
spring.kafka.dm.metadata.ssl-key-store-password	String	None	SSL keystore password for DM consumer

Property	Valid Values	Default Value	Descrip-tion
spring.kafka.dm.metadata.ssl-key-password	String	None	SSL key password for DM consumer
spring.kafka.dm.metadata.ssl.truststore.location	File path	None	Alternative SSL truststore location
spring.kafka.dm.metadata.ssl.truststore.password	String	None	Alternative SSL truststore password
spring.kafka.dm.metadata.ssl.truststore.type	String	JKS (Kafka default)	SSL truststore type for DM consumer
spring.kafka.dm.metadata.ssl.endpoint.identification.algorithm	String	None	Alternative SSL endpoint identification
spring.kafka.dm.metadata.ssl.protocol	String	TLSv1.3 (Kafka default)	SSL protocol for DM consumer
spring.kafka.dm.metadata.ssl.enabled.protocols	Comma-separated	TLSv1.3,TLSv1.2 (Kafka default)	Enabled SSL protocols for DM consumer
DM Consumer Security - SCRAM (Optional - Commented)			

Property	Valid Values	Default Value	Descrip-tion
spring.kafka.dm.metadata.sasl.mechanism	String	None	SASL mechanism for DM consumer SCRAM
spring.kafka.dm.metadata.sasl.jaas.config	JAAS config string	None	SASL JAAS for DM consumer SCRAM
spring.kafka.dm.metadata.client.dns.lookup	String	use_all_dns_ips (Kafka default)	DNS lookup strategy for DM consumer

10.7 Property Value Encryption

Refer to the Property Value Encryption section for more details.

SAPIENS

Partnering for Success

Contact Us

For more information, please visit or contact us at:

info.sapiens@sapiens.com

For Support related queries, contact us at:

Decision.Support@sapiens.com

For Documentation related queries, contact us at:

Decision_documentation@sapiens.com

www.sapiensdecision.com

Partnering for Success